



Commuting Conversions and Join Points for Call-by-Push-Value

JONATHAN CHAN, University of Pennsylvania, USA

MADI GUDIN*, Amherst College, USA

ANNABEL LEVY*, University of Maryland, Baltimore County, USA

STEPHANIE WEIRICH, University of Pennsylvania, USA

Levy's call-by-push-value (CBPV) is a language that subsumes both call-by-name and call-by-value lambda calculi by syntactically distinguishing values from computations and explicitly specifying execution order. This low-level handling of computation suspension and resumption makes CBPV suitable as a compiler intermediate representation (IR), while its substitution evaluation semantics affords compositional reasoning about programs. In particular, $\beta\eta$ -equivalences in CBPV have been used to justify compiler optimizations in low-level IRs. However, these equivalences do not validate *commuting conversions*, which are key transformations in compiler passes such as A-normalization. Such transformations syntactically rearrange computations without affecting evaluation order, and can reveal new opportunities for inlining.

In this work, we identify the commuting conversions of CBPV, define a *commuting conversion normal form* (CCNF) for CBPV, present a single-pass transformation into CCNF based on A-normalization, and prove that well-typed, translated programs evaluate to the same result. To avoid the usual code duplication issues that also arise with A-normal form, we adapt the explicit join point constructs by Maurer et al. [2017]. Our results are all mechanized in Lean 4.

CCS Concepts: • **Software and its engineering** → **Functional languages; Control structures.**

Additional Key Words and Phrases: commuting conversions, join points, call-by-push-value

ACM Reference Format:

Jonathan Chan, Madi Gudin, Annabel Levy, and Stephanie Weirich. 2026. Commuting Conversions and Join Points for Call-by-Push-Value. *Proc. ACM Program. Lang.* 10, OOPSLA1, Article 102 (April 2026), 25 pages. <https://doi.org/10.1145/3798210>

1 Introduction

Call-by-push-value (CBPV) [Levy 2003] is a programming language paradigm that syntactically distinguishes *values* from *computations*. Explicit constructs between values and computations encode evaluation order by marking where and when computations are suspended and resumed; as a result, CBPV subsumes call-by-name (CBN) and call-by-value (CBV) evaluation strategies. Levy also extends CBPV with various effects, isolating them to the computation fragment.

CBPV's distinction of pure values from effectful computations makes it a suitable compiler intermediate representation (IR) because it makes explicit many low-level considerations such as

* Both authors contributed equally to this work.

Authors' Contact Information: Jonathan Chan, University of Pennsylvania, Philadelphia, PA, USA, jcxz@seas.upenn.edu; Madi Gudin, Amherst College, Amherst, MA, USA, mgudin27@amherst.edu; Annabel Levy, University of Maryland, Baltimore County, Baltimore, MD, USA, alevy2@umbc.edu; Stephanie Weirich, University of Pennsylvania, Philadelphia, PA, USA, sweirich@seas.upenn.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/4-ART102

<https://doi.org/10.1145/3798210>

closures and control flow. Existing work covers CBPV as an IR to compile to, compile from, and perform compiler optimizations on.

- New [2019] draws parallels between how CBPV only binds values to variables with pushing and popping values on the stack in stack-based typed assembly language (STAL) [Morrisett et al. 2002], showing that they have comparable low-level features. He proposes adding instruction operations to CBPV, bringing it even closer to STAL.
- Explicit thunked computations in CBPV make it clear where closures ought to be created, and Sullivan et al. [2023] add abstract closures and incremental closure conversion to CBPV so that optimizations involving closure conversion can be done gradually.
- By binding all intermediate computations, CBPV makes low-level control flow explicit, but still lends to compositional reasoning with its substitution-based evaluation semantics. Garbuzov et al. [2018] therefore argue for using CBPV as a more compositional alternative to control flow graph (CFG) compiler representations by showing a tight equivalence between the two.
- While CFGs don't have a simple equational theory, Rizkallah et al. [2018] develop an equational theory for CBPV based on $\beta\eta$ -reductions. They use it to trivially justify compiler optimizations, such as dead branch elimination and constant folding, which would otherwise be more difficult to justify in CFGs directly.

However, $\beta\eta$ -equational theories as developed by Rizkallah et al. [2018] and later by Forster et al. [2019] do not validate *commuting conversion* compiler transformations, which syntactically rearrange computations without affecting their evaluation order. Forster et al. list a few commuting conversions in passing, but they aren't comprehensive and aren't framed in the context of compilation. Such transformations are key to compilation and form part of well-known passes such as A-normalization [Flanagan et al. 1993], which unnest computations so that control flow is syntactically explicit, making code easier to compile further. Moreover, these transformations can expose new opportunities for further program inlining and simplification. A known issue with naïvely applying commuting conversions, in particular in the presence of branching computations, is that code may be duplicated; applying many commuting conversions can then cause exponential code bloat. Maurer et al. [2017] resolve this with explicit join point constructs.

Therefore, we look at a source-to-source transformation that applies commuting conversions to CBPV, adapting its syntax and type system to accommodate explicit join points. Introducing separate constructs rather than using existing ones allows us to restrict them to only where they are needed, which can lead to more efficient, specialized compilation later on. In particular, because closures are represented by thunks, adding join points to CBPV reinforces the intent that join points represent local code blocks and should *not* be compiled to closures.

This work presents a normal form for CBPV with respect to commuting conversions, and a single-pass transformation into this normal form using join points. We prove that this transformation is type-preserving and evaluation-preserving: it does not change the meaning of CBPV programs. These proofs are mechanized in Lean 4 [de Moura et al. 2015] (v4.26.0), which are provided in the supplementary materials; definitions and theorems in this paper are accompanied by their corresponding file in the CBPV directory and by their name in the file. In the next section, we give an overview of the language and the transformation, leading to the following contributions.

- We identify a subset of CBPV that is normal with respect to all commuting conversions. To this subset, we add join point and jump constructs, and design a type system enforcing non-escaping join points and tail-only jumps. $\hookrightarrow$ Section 3
- We define a single-pass transformation from CBPV into our extended normal form, using joins and jumps to avoid duplicating computations. This transformation preserves well-typedness, proven straightforwardly by induction. $\hookrightarrow$ Section 4

- We show that the transformation preserves evaluation behaviour: a closed term returning a value with no thunks and its transformation must return the same terminal value. This is proven via a logical equivalence on terms, and requires showing that commuting conversions are logically equivalent. $\hookrightarrow$ [Section 5](#)

We discuss some other program equivalences and future work in [Section 6](#), and compare with related work in [Section 7](#).

2 Overview

The core ideas of this paper begin with the thesis that CBPV [[Levy 2003](#)] is suitable as a compiler IR because it represents control flow explicitly. In particular, it subsumes both CBN and CBV semantics: compiling the same lambda calculus term with the CBN or CBV compilation strategy yields different CBPV terms whose execution mirrors that of the original evaluation strategy.

$$\begin{aligned}
 v, w &::= x \mid () \mid \text{inl } v \mid \text{inr } v \mid \{m\} && \text{(values)} \\
 m, n &::= v! \mid \lambda x. m \mid n \mid v \mid \langle m, m \rangle \mid \text{fst } n \mid \text{snd } n \mid \text{return } v && \text{(computations)} \\
 &\mid \text{let } x \leftarrow n \text{ in } m \mid \text{case } v \text{ of } \{\text{inl } x \Rightarrow m; \text{inr } y \Rightarrow m\}
 \end{aligned}$$

Fig. 1. Syntax of call-by-push-value values and computations

CBPV syntactically distinguishes values and computations, listed in [Figure 1](#), using explicit thunks $\{m\}$ and forcing $v!$ to turn suspended computations into values and back, and explicit returns to embed values into computations, which can then be bound to a variable in a let expression. The CBN and CBV translations use thunks in different ways to enforce when computation occurs. Alongside the thunk and return constructs, we include the unit value, value sums, functions, and computation pairs.¹ In examples to follow, we also use boolean conditionals *if... then... else...* as syntactic sugar for case expressions on unit sums.

As an example of the way CBN and CBV translations differ, consider the lambda calculus term $(\lambda x. x)((\lambda y. y) z)$, which translates to the following two CBPV terms.

CBN $(\lambda x. x!) \{(\lambda y. y!) \{z!\}\}$

CBV $\text{let } f \leftarrow \text{return } \{\lambda x. \text{return } x\} \text{ in let } a \leftarrow (\text{let } g \leftarrow \text{return } \{\lambda y. \text{return } y\} \text{ in } g! z) \text{ in } f! a$

While both terms evaluate to the same final value z , their evaluation sequences are different. In the CBN translation, function arguments are thunked and passed wholesale, then forced as they are needed. In the CBV translation, the function and the argument are evaluated in order before carrying out the function application, using let bindings to express the explicit sequencing. First f is evaluated, followed by g , then by a , before the final application occurs.

Even though translating to CBPV has made control flow explicit, computations may still be arbitrarily nested, which hides some compiler optimizations. We look at commuting conversions next to handle such computations.

2.1 Commuting Conversions

Commuting conversions are syntactic transformations that swap nested eliminators while preserving evaluation order. An important benefit of performing commuting conversions is that it exposes inlining opportunities — that is, subexpressions that can be β -reduced to simplify code. For instance, we can commute a let-bound conditional by pushing the outer let expression into the inner conditional to reveal direct bindings of returned values.

¹ For simplicity, we omit value pairs, which are present in Levy's original CBPV; they are interesting in our setting because the eliminator is pattern matching, but we already have pattern matching on value sums to consider.

$$\begin{aligned}
& \text{let } b \leftarrow (\text{if } v \text{ then return false else return } w) \text{ in } m \\
& \implies \text{if } v \text{ then } (\text{let } b \leftarrow \text{return false in } m) \text{ else } (\text{let } b \leftarrow \text{return } w \text{ in } m) \quad [1]
\end{aligned}$$

Now we may choose to inline false inside of m in place of b and simplify the then branch. We may also choose *not* to inline w in the else branch if it happens that w is a particularly large value we don't want to duplicate. The inlining optimization wouldn't have been possible without the commutation, and is a well-known technique [Maurer et al. 2017].

Following Maurer et al. [2017], we define commuting conversions as all transformations that push elimination forms inside of tail positions, *i.e.* the bodies of let expressions and the branches of case expressions. Equation 1 pushes let bindings into conditionals; as shown below, we can also push let bindings into other let bindings, as well as function applications into let bindings and conditionals, to expose those inlining opportunities. Here, we use Barendregt's variable convention and assume that free variables in subterms are distinct from the variable bindings they get pushed under, noting that x and y are distinct variables.

$$\text{let } x \leftarrow (\text{let } y \leftarrow n \text{ in return } v) \text{ in } m \implies \text{let } y \leftarrow n \text{ in } (\text{let } x \leftarrow \text{return } v \text{ in } m) \quad [2]$$

$$(\text{let } x \leftarrow n \text{ in } \lambda y. m) v \implies \text{let } x \leftarrow n \text{ in } (\lambda y. m) v \quad [3]$$

$$(\text{if } w \text{ then } (\lambda x. m_1) \text{ else } (\lambda y. m_2)) v \implies \text{if } w \text{ then } (\lambda x. m_1) v \text{ else } (\lambda y. m_2) v \quad [4]$$

Another benefit of commuting conversions is that they unnest expressions, moving multiple steps of computation out of evaluation contexts, which brings them closer to lower-level code. Commuted code is easier to compile because control flow follows the shape of the syntax. In Equation 2, finding the next computation to execute on the left-hand side requires traversing into the x binding, then evaluating the y binding, and finally popping back out to evaluate the body m . In contrast, the right-hand side makes it explicit that we first evaluate the y binding, then the x binding, then the body.

This sequential nature of commuting conversions resembles the A-normalization compiler pass for the lambda calculus into A-normal form (ANF) because commuting conversions are part of the A-reductions that characterize ANF [Flanagan et al. 1993]. A-normalization makes control flow syntactically explicit by binding intermediate computations and by sequentializing computations. Translations from the lambda calculus to CBPV already bind intermediate computations, so what remains is to sequentialize them via commuting conversions. Rather than performing commuting conversions one by one, which examines the whole program each time, we present a single-pass transformation into commuting conversion normal form (CCNF) in Section 4 that resembles the usual single-pass transformation into ANF.

$$\begin{aligned}
n &::= v! \mid \lambda x. m \mid n v \mid \text{return } v \mid \langle m, m \rangle \mid \text{fst } n \mid \text{snd } n && \text{(tail-free computations)} \\
m &::= n \mid \text{let } x \leftarrow n \text{ in } m \mid \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} && \text{(configurations)}
\end{aligned}$$

Fig. 2. Commuting conversion normal form of computations

In ANF, terms are divided into values, computations, and configurations, where computations are a subset of configurations that don't include let expressions and conditionals (or case expressions generally). Figure 2 makes the same distinction for CBPV, where values remain unchanged; to avoid confusion, we also call the latter *configurations*, while we call the former *tail-free computations*, which are both subsets of CBPV *computations*. Because commuting conversions push computations only into tail positions, new opportunities for inlining only occur in m positions, so performing those new inlinings won't violate CCNF.

2.2 Join Points

Generalizing [Equation 1](#) to case expressions and arbitrary computations, the corresponding commuting conversion pushes let bindings into case branches.

$$\begin{aligned} & \text{let } x \leftarrow (\text{case } v \text{ of } \{\text{inl } y_1 \Rightarrow n_1; \text{inr } y_2 \Rightarrow n_2\}) \text{ in } m \\ \Rightarrow & \text{case } v \text{ of } \{\text{inl } y_1 \Rightarrow (\text{let } x \leftarrow n_1 \text{ in } m); \text{inr } y_2 \Rightarrow (\text{let } x \leftarrow n_2 \text{ in } m)\} \end{aligned} \quad [5]$$

There is a code optimization issue with [Equations 1](#) and [5](#): the let body m gets duplicated across the branches. If the size of m is very large, this can cause code bloat, especially if the branches contain further case expressions. The usual solution for the lambda calculus with let expressions is to let-bind a closure containing m to be called at the end of the branch, also known as a *join point*. Similarly, in CBPV, we can bind a thunked function to be forced and applied.

$$\begin{aligned} & \text{let } x \leftarrow (\text{case } v \text{ of } \{\text{inl } y_1 \Rightarrow m_1; \text{inr } y_2 \Rightarrow m_2\}) \text{ in } m \\ \Rightarrow & \text{let } z \leftarrow \text{return } \{\lambda x. m\} \text{ in} \\ & \text{case } v \text{ of } \{\text{inl } y_1 \Rightarrow (\text{let } x \leftarrow m_1 \text{ in } z! x); \text{inr } y_2 \Rightarrow (\text{let } x \leftarrow m_2 \text{ in } z! x)\} \end{aligned} \quad [6]$$

To the next compiler passes that see this code, the thunk is a value that may capture variables and escape its scope, so somewhere along the pipeline the thunk will be converted into a closure and lifted out, and $z! x$ will correspond to a function call. However, we know from the commuting conversion that the thunk will never escape its scope, since it's never passed to a function or stored in another thunk; all that we want to do is jump to the m within and pop an argument x . Its purpose is only to join up branches of a computation, and should be compiled to a local code block.

$$m ::= \dots \mid \text{join } j \ x = m \text{ in } m \mid \text{jump } j \ v \quad (\text{join points, jumps})$$

Fig. 3. Extended configurations

Inspired by [Maurer et al. \[2017\]](#), who tackle the same issue with commuting conversions in System F with case expressions, we add explicit join point and jump constructs to CBPV in [Figure 3](#). They are accompanied by typing rules that restrict where join points may be used, which we cover in [Section 3](#). In contrast to *op. cit.*, our jumps aren't tail-free computations and may only appear in tail position, which simplifies both the evaluation semantics and our proofs. [Section 7](#) discusses why we can make this restriction.

Coming back to the commuting conversion of [Equation 6](#), we use join points in place of the bound thunk, jumping to them after binding the branches. Let expressions only bind a single variable, so we only need join points and jumps that take a single argument.

$$\begin{aligned} & \text{let } x \leftarrow (\text{case } v \text{ of } \{\text{inl } y_1 \Rightarrow m_1; \text{inr } y_2 \Rightarrow m_2\}) \text{ in } m \\ \Rightarrow & \text{join } j \ x = m \text{ in} \\ & \text{case } v \text{ of } \{\text{inl } y_1 \Rightarrow (\text{let } x \leftarrow m_1 \text{ in jump } j \ x); \\ & \text{inr } y_2 \Rightarrow (\text{let } x \leftarrow m_2 \text{ in jump } j \ x)\} \end{aligned} \quad [7]$$

What about duplicated values? The commuting conversion in [Equation 4](#), which pushes applications into case expressions, similarly duplicates the application argument. However, values can be easily deduplicated using the existing let binding.

$$(\text{if } w \text{ then } m_1 \text{ else } m_2) \ v \Rightarrow \text{let } x \leftarrow \text{return } v \text{ in } (\text{if } w \text{ then } m_1 \ x \text{ else } m_2 \ x) \quad [8]$$

Deduplicating values potentially optimizes allocations, but as this is orthogonal to control flow, we consider it outside the scope of this work.

$A ::= \top \mid A + A \mid \cup B$	(value types)
$B ::= A \rightarrow B \mid B \& B \mid F A$	(computation types)
$v, w ::= x \mid () \mid \text{inl } v \mid \text{inr } v \mid \{m\}$	(values)
$m, n ::= v! \mid \lambda x. m \mid n \ v \mid \langle m, m \rangle \mid \text{fst } n \mid \text{snd } n \mid \text{return } v$	(computations)
$\mid \text{let } x \leftarrow n \text{ in } m \mid \text{case } v \text{ of } \{\text{inl } x \Rightarrow m; \text{inr } y \Rightarrow m\}$	
$\mid \text{join } j \ x = m \text{ in } m \mid \text{jump } j \ v$	

Fig. 4. Syntax of value, computations, and their types [Syntax:ValType,ComType,Val,Com]

3 CBPV with Join Points

While [Section 2](#) presents the source (plain CBPV) and target (CCNF CBPV with join points) languages with distinct syntactic forms, in our mechanization (and thus our technical presentation here), we use a single unified syntax and treat CC-normalization as a source-to-source translation, showing *a posteriori* in [Section 4](#) that the output of the translation satisfies [Figures 2 and 3](#). In [Section 5](#), we reason about equivalence between a CBPV term and its translation, which requires both sides of the equivalence to belong to the same syntactic category.

[Figure 4](#) lists the full syntax of values and computations, along with value types and computation types. The new syntactic forms not found in plain CBPV are highlighted in salmon. For clarity, although the mechanization uses de Bruijn indexing and simultaneous substitutions, we present the syntax here in nominal form, with $[v\{x \mapsto w\}]$, $[m\{x \mapsto w\}]$ denoting single (capture-avoiding) substitution of x for w in v and m , respectively. The syntax and related definitions in the mechanization are also intrinsically well scoped with respect to jump variables, which we omit here, and freely use jump well-scopedness throughout the proofs.

3.1 Evaluation Semantics

The purpose of join points is best explained by what they do; their single-step evaluation rules are listed in [Figure 5](#), alongside rules for the usual CBPV constructs. We write $[m \rightsquigarrow^* m']$ for the reflexive, transitive closure of evaluation. The names of the rules involving join points and jumps not found in plain CBPV are highlighted in salmon.

The usual rules [E-FORCE](#) through [E-RIGHT](#) say that thunks get forced to their inner computations, applied functions substitute in their arguments, the first and second components of a computational pair can be projected out, and case analysis on the left or right injections reduce to the left or right branches, respectively. Rule [E-CONG](#) states that evaluation may occur under evaluation contexts E , which are elimination forms with holes in scrutinee position. We exclude join expressions from evaluation contexts because we later use E to define our set of commuting conversions, which don't commute into join expressions.

Join expressions evaluate under a context $\text{join } j \ x = m \text{ in } \square$ in rule [E-JOIN](#). The inner evaluation is done when it reaches a jump or a terminal computation tm , which are computation introduction forms. If the body of the join expression is a jump to the bound join point, then the value jumped with is substituted into the join point in rule [E-JUMP](#). If it jumps to a join point further outward, then this inner join point binding is discarded in rule [E-SKIP](#). The binding is also discarded when a terminal is reached in rule [E-DROP](#).

To illustrate, consider the following reduction sequence for three join points. Intuitively, we jump to j_3 which jumps to j_1 which is m_1 , so we end up at $m_1\{x \mapsto v\}$; the reductions that apply to get there are first to jump, then to skip, then to jump.

$$\boxed{m \rightsquigarrow m}$$

$$\begin{array}{ll}
\{m\}! \rightsquigarrow m & \text{(E-FORCE)} \\
(\lambda x. m) v \rightsquigarrow m\{x \mapsto v\} & \text{(E-APP)} \\
\text{fst } \langle m_1, m_2 \rangle \rightsquigarrow m_1 & \text{(E-FST)} \\
\text{snd } \langle m_1, m_2 \rangle \rightsquigarrow m_2 & \text{(E-SND)} \\
\text{let } x \leftarrow \text{return } v \text{ in } m \rightsquigarrow m\{x \mapsto v\} & \text{(E-RET)} \\
\text{case (inl } v) \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} \rightsquigarrow m_1\{x \mapsto v\} & \text{(E-LEFT)} \\
\text{case (inr } v) \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} \rightsquigarrow m_2\{y \mapsto v\} & \text{(E-RIGHT)} \\
\text{join } j \ x = m \text{ in jump } j \ v \rightsquigarrow m\{x \mapsto v\} & \text{(E-JUMP)} \\
\text{join } j' \ x = m' \text{ in jump } j \ v \rightsquigarrow \text{jump } j \ v & \text{(E-SKIP)} \\
\text{join } j' \ x = m' \text{ in } tm \rightsquigarrow tm & \text{(E-DROP)} \\
\\
\frac{m_1 \rightsquigarrow m_2}{E[m_1] \rightsquigarrow E[m_2]} \text{E-CONG} & \frac{m_1 \rightsquigarrow m_2}{\text{join } j \ x = m \text{ in } m_1 \rightsquigarrow \text{join } j \ x = m \text{ in } m_2} \text{E-JOIN}
\end{array}$$

where $E ::= \text{let } x \leftarrow \square \text{ in } m \mid \square \ v \mid \text{fst } \square \mid \text{snd } \square$ (evaluation contexts)
 $tm ::= \lambda x. m \mid \text{return } v \mid \langle m, m \rangle$ (terminal computations)

Fig. 5. Evaluation rules for computations [Evaluation:Eval,nf]

$$\begin{array}{ll}
\text{join } j_1 \ x = m_1 \text{ in join } j_2 \ x = m_2 \text{ in join } j_3 \ x = \text{jump } j_1 \ x \text{ in jump } j_3 \ v & \\
\rightsquigarrow \text{join } j_1 \ x = m_1 \text{ in join } j_2 \ x = m_2 \text{ in jump } j_1 \ v & \text{(E-JUMP)} \\
\rightsquigarrow \text{join } j_1 \ x = m_1 \text{ in jump } j_1 \ v & \text{(E-SKIP)} \\
\rightsquigarrow m_1\{x \mapsto v\} & \text{(E-JUMP)}
\end{array}$$

This is in fact the *only* sequence of reductions, because evaluation is deterministic. If we define normalization as evaluation to a terminal,² then normalization is deterministic as well, and multi-step evaluations always evaluate to the same terminal.

LEMMA 3.1 (DETERMINISM OF EVALUATION). [Evaluation:Eval.det]
If $m \rightsquigarrow m_1$ and $m \rightsquigarrow m_2$, then $m_1 = m_2$.

Definition 3.2 (Normalization). [Evaluation:Norm]
 m normalizes to tm , written as $\boxed{m \Downarrow tm}$, if $m \rightsquigarrow^* tm$.

COROLLARY 3.3 (DETERMINISM OF NORMALIZATION). [Evaluation:Norm.join]
If $m \Downarrow tm_1$ and $m \Downarrow tm_2$, then $tm_1 = tm_2$.

COROLLARY 3.4 (MERGING). [Evaluation:Evals.merge] *If $m \Downarrow tm$ and $m \rightsquigarrow^* m'$, then $m' \Downarrow tm$.*

² This is weak normalization, not strong normalization, since subterms are not normalized. We won't consider strong normalization, so we simply call it normalization.

$\boxed{\Gamma \vdash v : A}$	$\boxed{\Gamma \mid \Delta \vdash m : B}$			
T-VAR $\frac{x : A \in \Gamma}{\Gamma \vdash x : A}$	T-UNIT $\frac{}{\Gamma \vdash () : \top}$	T-LEFT $\frac{\Gamma \vdash v : A_1}{\Gamma \vdash \text{inl } v : A_1 + A_2}$	T-RIGHT $\frac{\Gamma \vdash v : A_2}{\Gamma \vdash \text{inr } v : A_1 + A_2}$	T-THUNK $\frac{\Gamma \mid \cdot \vdash m : B}{\Gamma \vdash \{m\} : \cup B}$
T-FORCE $\frac{\Gamma \vdash v : \cup B}{\Gamma \mid \Delta \vdash v! : B}$	T-FUN $\frac{\Gamma, x : A \mid \cdot \vdash m : B}{\Gamma \mid \Delta \vdash \lambda x. m : A \rightarrow B}$	T-APP $\frac{\Gamma \mid \cdot \vdash n : A \rightarrow B \quad \Gamma \vdash v : A}{\Gamma \mid \Delta \vdash n v : B}$		
T-RET $\frac{\Gamma \vdash v : A}{\Gamma \mid \Delta \vdash \text{return } v : F A}$	T-LET $\frac{\Gamma \mid \cdot \vdash n : F A \quad \Gamma, x : A \mid \Delta \vdash m : B}{\Gamma \mid \Delta \vdash \text{let } x \leftarrow n \text{ in } m : B}$			
T-CASE $\frac{\Gamma \vdash v : A_1 + A_2 \quad \Gamma, x : A_1 \mid \Delta \vdash m_1 : B \quad \Gamma, y : A_2 \mid \Delta \vdash m_2 : B}{\Gamma \mid \Delta \vdash \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} : B}$				
T-PAIR $\frac{\Gamma \mid \cdot \vdash m_1 : B_1 \quad \Gamma \mid \cdot \vdash m_2 : B_2}{\Gamma \mid \Delta \vdash \langle m_1, m_2 \rangle : B_1 \& B_2}$	T-FST $\frac{\Gamma \mid \cdot \vdash n : B_1 \& B_2}{\Gamma \mid \Delta \vdash \text{fst } n : B_1}$	T-SND $\frac{\Gamma \mid \cdot \vdash n : B_1 \& B_2}{\Gamma \mid \Delta \vdash \text{snd } n : B_2}$		
T-JOIN $\frac{\Gamma, x : A \mid \Delta \vdash m_1 : B \quad \Gamma \mid \Delta, j : A \hat{\wedge} B \vdash m_2 : B}{\Gamma \mid \Delta \vdash \text{join } j x = m_1 \text{ in } m_2 : B}$	T-JUMP $\frac{j : A \hat{\wedge} B \in \Delta \quad \Gamma \vdash v : A}{\Gamma \mid \Delta \vdash \text{jump } j v : B}$			

Fig. 6. Typing rules for values and computations [Typing: ValWt, ComWt]

3.2 Typing Rules

By adding join points, we now have two different forms of bindings: value variables x bind values and have value types, while jump variables j bind join points, which are computations of type B that take some value argument of type A . As the bindings have different types, we use two different typing contexts: $\Gamma ::= \cdot \mid \Gamma, x : A$ for value contexts, and $\Delta ::= \cdot \mid \Delta, j : A \hat{\wedge} B$ for jump contexts.

Without the jump contexts, rules **T-VAR** through **T-SND** in Figure 6 are the usual typing rules for the values and computations of plain CBPV. When we include jump contexts, they are either threaded through rules unchanged, or they are explicitly empty in some premises. In particular, joins represent local blocks of code that are jumped to with the current environment, and aren't closures. The computation inside of a thunk in rule **T-THUNK** needs to be closed with respect to jump variables to prevent jumps from escaping their local scopes.

Furthermore, we restrict jumps to tail positions in contrast to Maurer et al. [2017], who allow jumps in scrutinee positions. They require type polymorphism to assign arbitrary types to jumps so that type safety isn't violated. We propagate the jump context only in premises with the same type so that jumping never changes the type. Although this restricts jumps to only tail positions, it exactly captures what CC-normalization needs: the ability to jump to another computation at the very end of the previous one. As a result, the evaluation semantics are simpler and the metatheory is cleaner. To enforce this restriction, the scrutinee premises of rules **T-APP**, **T-LET**, **T-FST**, and **T-SND** require empty jump contexts, while the tail premises of rules **T-LET** and **T-CASE** may contain jumps.

Additionally, we don't allow computation constructors rules **T-FUN** and **T-PAIR** to contain jumps, which prevents stuck terms such as $\text{fst } (\text{join } j \ x = m \text{ in } \langle \text{jump } j \ v, \text{jump } j \ w \rangle)$. While the evaluation rules can be extended so that this computation reduces to $m\{x \mapsto v\}$, doing so again complicates both the evaluation semantics and the metatheory, and join points for commuting conversions don't require such flexibility.

In rule **T-JOIN**, we extend Δ with a join declaration when checking the body m_2 , which may jump to the join point m_1 . Both have the same type B because jumping in tail position doesn't change the type, and rule **T-JUMP** indicates that jumping to the join point at j with a value of type A indeed has the same B . This is why the jump context tracks the type B , and having the same type will be crucial later for the logical equivalence in [Section 5.1](#).

We can then, for example, jump to a join point in only one branch of a case expression so long as the join point has the same type as the other branch, as in the following derivable judgement, which we typeset with a dashed bar.

$$\frac{\Gamma \vdash v : A_1 + A_2 \quad \Gamma, x : A_1 \mid \cdot \vdash m_1 : B \quad \Gamma, y : A_2 \mid \cdot \vdash m_2 : B}{\Gamma \mid \cdot \vdash \text{join } j \ x = m_1 \text{ in case } v \text{ of } \{\text{inl } x \Rightarrow \text{jump } j \ x; \text{inr } y \Rightarrow m_2\} : B}$$

The important typing lemmas that we need are weakening lemmas for both value and jump contexts. In the mechanization, they are proven by induction on the typing derivation via renaming lemmas; for now, we ignore issues of renaming de Bruijn indices, as they are standard. We don't need syntactic progress and preservation lemmas, as type safety will follow from our semantic proofs as [Corollary 5.10](#).

LEMMA 3.5 (WEAKENING (VALUE CONTEXTS)). [[Typing:wtWeakenVal](#), [wtWeakenCom](#)]
Suppose x is not free in v , m . If $\Gamma_1, \Gamma_2 \vdash v : A$, then $\Gamma_1, x : A', \Gamma_2 \vdash v : A$, and if $\Gamma_1, \Gamma_2 \mid \Delta \vdash m : B$, then $\Gamma_1, x : A, \Gamma_2 \mid \Delta \vdash m : B$.

LEMMA 3.6 (WEAKENING (JUMP CONTEXTS)). [[Typing:wtWeakenJ](#)]
Suppose j is not free in m . If $\Gamma \mid \Delta_1, \Delta_2 \vdash m : B$, then $\Gamma \mid \Delta_1, j : A \nearrow B', \Delta_2 \vdash m : B$.

4 Commuting Conversion Normalization

In [Section 2](#), we listed four examples of commuting conversions as [Equations \(1\) to \(4\)](#). The general formulation commutes evaluation contexts with *tail contexts*, given below, which are elimination forms with holes where computations continue, which in our case are the bodies of let expressions and the branches of case expressions. We can think of computations with a tail as those that have a “next step”. As with evaluation contexts, we exclude join point expressions from tail contexts because our source language is plain CBPV with no join points or jumps.

$$L ::= \text{let } x \leftarrow n \text{ in } \square \mid \text{case } v \text{ of } \{\text{inl } x \Rightarrow \square; \text{inr } y \Rightarrow \square\} \quad (\text{tail contexts})$$

The commuting conversions we consider can then be stated as $E[L[m]] \Rightarrow L[E[m]]$. Informally, they unnest code because they move evaluation contexts into the “next step”, and they expose inlining opportunities because these evaluation contexts are no longer blocked by tail contexts that still have a “first step” to compute. The shape of commuting conversions inform the shape of the normal form of computations, reproduced in [Figure 7](#), where tail-free computations n are computations that don't contain tail contexts, and configurations m are all computations that don't contain tail contexts in scrutinee positions, *i.e.* where holes appear in evaluation contexts. By inspection, CCNF is indeed normal with respect to commuting conversions because no ms appear in ns , and therefore there must be no more commuting conversions to do.

To transform a plain CBPV program to one in CCNF, we follow [Flanagan et al. \[1993\]](#) and define a compiler using a continuation K , whose forms are given below. As usual, it can be the empty

$$\begin{aligned}
v &::= x \mid () \mid \text{inl } v \mid \text{inr } v \mid \{m\} && \text{(values)} \\
n &::= v! \mid \lambda x. m \mid n \ v \mid \text{return } v \mid \langle m, m \rangle \mid \text{fst } n \mid \text{snd } n && \text{(tail-free computations)} \\
m &::= n \mid \text{let } x \leftarrow n \text{ in } m \mid \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} && \text{(configurations)} \\
&\quad \mid \text{join } j \ x = m_1 \text{ in } m_2 \mid \text{jump } j \ v
\end{aligned}$$

Fig. 7. Commuting conversion normal form with join points [CCNF:isVal,isCom,isCfg]

continuation $\square$, or it can be the let continuation $\text{let } x \leftarrow \square \text{ in } m$, which expects to bind some tail-free computation and continues on with a configuration m .

$$K ::= \square \mid \text{let } x \leftarrow \square \text{ in } m \mid K :: k \quad k ::= \square \ v \mid \text{fst } \square \mid \text{snd } \square \quad [\text{CCNF:K}]$$

However, we have three more continuation forms corresponding to each of the remaining three evaluation contexts: application, first projection, and second projection. In contrast to ANF for the lambda calculus, functions and computation pairs are *computations* and not *values*, so they can't be let-bound and their elimination forms can take arbitrary tail-free computations.

$$\begin{aligned}
(\text{let } x \leftarrow \square \text{ in } m)[n] &::= \text{let } x \leftarrow n \text{ in } m & (K :: \text{fst } \square)[n] &::= K[\text{fst } n] & \square[n] &::= n \\
(K :: \square \ v)[n] &::= K[n \ v] & (K :: \text{snd } \square)[n] &::= K[\text{snd } n]
\end{aligned}$$

Fig. 8. Plugging a computation in a continuation [CCNF:plug]

A continuation K can be combined with a computation n by the plugging operation $\overline{K[n]}$, which collapses the stack of ks in the continuation and replaces the final hole $\square$ by n , defined in Figure 8. Although the operation is defined over all computations, as the holes only appear in n positions, we can only plug tail-free computations into continuations if we want to produce a configuration.

$$\begin{aligned}
& \llbracket v! \rrbracket K := K[\llbracket v \rrbracket!] \\
& \llbracket \text{return } v \rrbracket K := K[\text{return } \llbracket v \rrbracket] \\
& \llbracket \lambda x. m \rrbracket K := K[\lambda x. \llbracket m \rrbracket \square] \\
& \llbracket \langle m_1, m_2 \rangle \rrbracket K := K[\langle \llbracket m_1 \rrbracket \square, \llbracket m_2 \rrbracket \square \rangle] \\
& \llbracket m \ v \rrbracket K := \llbracket m \rrbracket (K :: \square \ v) \\
& \llbracket \text{fst } m \rrbracket K := \llbracket m \rrbracket (K :: \text{fst } \square) \\
& \llbracket \text{snd } m \rrbracket K := \llbracket m \rrbracket (K :: \text{snd } \square) \\
& \llbracket \{m\} \rrbracket := \{\llbracket m \rrbracket \square\} \quad \llbracket \text{let } x \leftarrow m_1 \text{ in } m_2 \rrbracket K := \llbracket m_1 \rrbracket (\text{let } x \leftarrow \square \text{ in } \llbracket m_2 \rrbracket K)
\end{aligned}$$

(a) Translation of values [CCNF:CCval] (b) Translation of computations [CCNF:CCval]

Fig. 9. Commuting conversion normalization translation (excluding case)

The translation of values $\llbracket v \rrbracket$ and computations $\llbracket m \rrbracket K$ are mutually defined in Figure 9; we delay the translation of case expressions for the moment. The translation of computations takes a continuation as a second argument, representing the rest of the computation that expects the result of the translated computation. The translation of values is directly recursive on the term, with the translation of thunks being the thunk of the translated computation using the empty continuation, since there's no computation left to do inside the thunk.

In the translation of let expressions, the evaluation order would first compute m_1 , then m_2 , followed by whatever computation remains in K . Therefore, we translate m_1 using a let continuation that represents binding the result of m_1 to x then running the translation of m_2 . For function

applications and pair projections, we translate the subterm, which we expect to yield a function or a pair, under the continuation extended with application or projection, respectively.

To see how the translation yields CCNFs, we can look at how they act on the left-hand side of the commuting conversions in Equations 2 and 3, assuming that n is already in CCNF, and using the fact that $\llbracket n \rrbracket K = K[n]$.

$$\begin{aligned}
 & \llbracket \text{let } x \leftarrow (\text{let } y \leftarrow n \text{ in return } v) \text{ in } m \rrbracket \square && \text{(Equation 2)} \\
 & = \llbracket \text{let } y \leftarrow n \text{ in return } v \rrbracket (\text{let } x \leftarrow \square \text{ in } \llbracket m \rrbracket \square) && (\text{let}) \\
 & = \llbracket n \rrbracket (\text{let } y \leftarrow \square \text{ in } \llbracket \text{return } v \rrbracket (\text{let } x \leftarrow \square \text{ in } \llbracket m \rrbracket \square)) && (\text{let}) \\
 & = (\text{let } y \leftarrow \square \text{ in } (\text{let } x \leftarrow \square \text{ in } \llbracket m \rrbracket \square) [\text{return } \llbracket v \rrbracket]) [n] && (n, \text{ret}) \\
 & = \text{let } y \leftarrow n \text{ in } (\text{let } x \leftarrow \text{return } \llbracket v \rrbracket \text{ in } \llbracket m \rrbracket \square) && (\text{plug}) \\
 & \llbracket (\text{let } x \leftarrow n \text{ in } \lambda y. m) v \rrbracket \square && \text{(Equation 3)} \\
 & = \llbracket \text{let } x \leftarrow n \text{ in } \lambda y. m \rrbracket (\square \llbracket v \rrbracket) && (\text{app}) \\
 & = \llbracket n \rrbracket (\text{let } x \leftarrow \square \text{ in } \llbracket \lambda y. m \rrbracket (\square \llbracket v \rrbracket)) && (\text{let}) \\
 & = (\text{let } x \leftarrow \square \text{ in } (\square \llbracket v \rrbracket) [\lambda y. \llbracket m \rrbracket \square]) [n] && (n, \text{fun}) \\
 & = \text{let } x \leftarrow n \text{ in } (\lambda y. \llbracket m \rrbracket \square) \llbracket v \rrbracket && (\text{plug})
 \end{aligned}$$

Branching and join points. The naïve translation of conditionals duplicates the continuation:

$$\llbracket \text{if } v \text{ then } m_1 \text{ else } m_2 \rrbracket K = \text{if } \llbracket v \rrbracket \text{ then } \llbracket m_1 \rrbracket K \text{ else } \llbracket m_2 \rrbracket K.$$

If K only contains application and projection continuations, this is the desired translation. From $\text{fst}(\text{if } v \text{ then } \langle m_1, m_2 \rangle \text{ else } n)$ we get $\text{if } v \text{ then } \text{fst} \langle m_1, m_2 \rangle \text{ else } \text{fst } n$, whose true branch can then be inlined. It doesn't make sense to jump to a join point that performs the projection, since it would make the inlining opportunity harder to find, and require unnecessarily thinking the computation pair to pass it to the join point.

If the continuation is $\text{let } x \leftarrow \square \text{ in } m$, duplicating it would duplicate the arbitrary configuration m . In this case, we move m to a join point and replace it in the continuation by a jump. For example, from $\text{let } x \leftarrow (\text{if } v \text{ then return } w \text{ else } n) \text{ in } m$ we get

$$\text{join } j \ x = m \text{ in } (\text{if } v \text{ then } (\text{let } x \leftarrow \text{return } w \text{ in jump } j \ x) \text{ else } (\text{let } x \leftarrow n \text{ in jump } j \ x)),$$

and the true branch can still inline the let binding. However, we avoid creating extra join points in nested branching. For example, from $\text{let } x \leftarrow (\text{if } v \text{ then } n_1 \text{ else } (\text{if } w \text{ then } n_2 \text{ else } n_3)) \text{ in } m$ we get

$$\begin{aligned}
 & \text{join } j \ x = m \text{ in} \\
 & \quad \text{if } v \text{ then } (\text{let } x \leftarrow n_1 \text{ in jump } j \ x) \text{ else} \\
 & \quad \quad (\text{if } w \text{ then } (\text{let } x \leftarrow n_2 \text{ in jump } j \ x) \text{ else } (\text{let } x \leftarrow n_3 \text{ in jump } j \ x)),
 \end{aligned}$$

rather than another extraneous join point $j' \ x = \text{jump } j \ x$ in the outer false branch.

Figure 10 generalizes this strategy to case expressions and nested continuations; whether we construct a join point depends on whether the continuation ends in a let continuation whose body isn't already a jump to a join point. In the mechanization, this is a source-to-source translation over CBPV terms (without join points) to CBPV terms (with join points). We need to explicitly prove that the translation indeed produces terms in CCNF, which holds by inspection, *i.e.* mutual induction over the syntax.

LEMMA 4.1 (PLUGGING PRESERVES CCNF). $[\text{CCNF} : \text{isK}.\text{plug}]$

If the subterms of K are in CCNF and n is in CCNF, then $K[n]$ is in CCNF.

$$\begin{aligned}
& \llbracket \text{case } v \text{ of } \{ \text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2 \} \rrbracket K := \\
& \left\{ \begin{array}{ll}
\text{case } \llbracket v \rrbracket \text{ of } \{ \text{inl } y \Rightarrow \llbracket m_1 \rrbracket K; \text{inr } z \Rightarrow \llbracket m_2 \rrbracket K \} & \text{if } K \equiv \square :: \dots :: k_i \\
& \text{or } K \equiv (\text{let } x \leftarrow \square \text{ in jump } j \ x) :: \dots :: k_i \\
\text{join } j \ x = m \text{ in} & \text{if } K \equiv (\text{let } x \leftarrow \square \text{ in } m) :: \dots :: k_i \\
\text{case } \llbracket v \rrbracket \text{ of } \{ \text{inl } y \Rightarrow \llbracket m_1 \rrbracket K'; \text{inr } z \Rightarrow \llbracket m_2 \rrbracket K' \} & \text{and } m \neq \text{jump } j \ x
\end{array} \right. \\
& \text{where } K' := (\text{let } x \leftarrow \square \text{ in jump } j \ x) :: \dots :: k_i
\end{aligned}$$

Fig. 10. Translation of case expressions to CCNF with join points [CCNF:CCcom,K.jumpify]

LEMMA 4.2 (CCNF PRESERVATION). [CCNF:isCCNF,isK.jumpify]
If the subterms of K are in CCNF, then $\llbracket v \rrbracket$ and $\llbracket m \rrbracket K$ are in CCNF.

4.1 Commuting Conversion Preserves Typing

With Lemma 4.2, we know that the translation performs all the commuting conversions in the directions we desire. However, this alone doesn't guarantee that the translation preserves the *meaning* of the values and computations. We say what we mean by meaning preservation in the next section, whose proof uses typing preservation: a translated term must behave like the same kind of term as the original term.

$$\boxed{\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2}$$

$$\begin{array}{c}
\text{K-LET} \quad \frac{\Gamma, x : A \mid \Delta \vdash m : B}{\Gamma \mid \Delta \vdash \text{let } x \leftarrow \square \text{ in } m : F \ A \Rightarrow B} \qquad \text{K-APP} \quad \frac{\Gamma \vdash v : A \quad \Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2}{\Gamma \mid \Delta \vdash K :: \square \ v : (A \rightarrow B_1) \Rightarrow B_2} \\
\\
\text{K-FST} \quad \frac{\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B}{\Gamma \mid \Delta \vdash K :: \text{fst } \square : (B_1 \ \& \ B_2) \Rightarrow B} \qquad \text{K-SND} \quad \frac{\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B}{\Gamma \mid \Delta \vdash K :: \text{snd } \square : (B_1 \ \& \ B_2) \Rightarrow B} \qquad \text{K-HOLE} \quad \frac{}{\Gamma \mid \Delta \vdash \square : B \Rightarrow B}
\end{array}$$

Fig. 11. Typing rules for continuations [CCNF:wtK]

To show type preservation, we first need a typing judgement for the continuations, given in Figure 11. A continuation K having type $B_1 \Rightarrow B_2$ means that if the hole in K represents a missing computation of type B_1 , then K with its hole plugged in would be a computation of type B_2 . Consequently, we can show that plugging preserves typing, which is used in the proof of type preservation. We also need an inversion lemma for continuations that end in let continuations.

LEMMA 4.3 (PLUGGING PRESERVES TYPING). [CCNF:wtK.plug]
If $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$ and $\Gamma \mid \cdot \vdash n : B_1$, then $\Gamma \mid \Delta \vdash K[n] : B_2$.

PROOF. By induction on the typing derivation of K , using Lemma 3.6 in the K-HOLE case. $\square$

LEMMA 4.4 (LET CONTINUATION INVERSION). [CCNF:wtK.jumpify]
If $K = (\text{let } x \leftarrow \square \text{ in } m) :: \dots :: k_i$ and $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$, then there exists some A such that $\Gamma \mid \Delta, j : A \nearrow B_2 \vdash K' : B_1 \Rightarrow B_2$ and $\Gamma, x : A \mid \Delta \vdash m : B_2$, where $K' = (\text{let } x \leftarrow \square \text{ in jump } j \ x) :: \dots :: k_i$.

PROOF. By induction on the typing derivation of K . $\square$

LEMMA 4.5 (TYPE PRESERVATION). [CCNF:preservation] Suppose v and m are plain CBPV terms (and thus have no join points or jumps). If $\Gamma \vdash v : A$, then $\Gamma \vdash \llbracket v \rrbracket : A$, and if $\Gamma \mid \cdot \vdash m : B_1$ and $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$, then $\Gamma \mid \Delta \vdash \llbracket m \rrbracket K : B_2$.

PROOF. By mutual induction on the typing derivations of v and m , using Lemma 4.3 in the T-FORCE, T-FUN, and T-PAIR cases. In the T-CASE case, if K is a let continuation, use Lemma 4.4 and rule T-JOIN to construct the derivation. $\square$

5 Commuting Conversion Normalization Preserves Evaluation

Normalizing by all commuting conversions shouldn't affect the meaning of a program. Formally, if a closed computation evaluates to a returned value, then its translation runs to the same value. Because we don't evaluate inside of thunks, we consider only computations that return *ground* values, which are those of type $T ::= \top \mid T + T$. Otherwise, a thunk and its CCNF are values that don't evaluate any further, but aren't necessarily syntactically equal.

THEOREM 5.1. Given m such that $\cdot \mid \cdot \vdash m : F T$, if $m \Downarrow$ return v , then $\llbracket m \rrbracket \square \Downarrow$ return v .

We prove this property as a corollary of a logical equivalence between a computation and its translation. This machinery is required because the simpler of method using a simulation argument, such as the following statement, unfortunately doesn't work.

FALSEHOOD 5.1 (SIMULATION). If $m \rightsquigarrow n$ then $\llbracket m \rrbracket \square \rightsquigarrow^* \llbracket n \rrbracket \square$.

COUNTEREXAMPLE. Suppose we have tail-free computations n_1, n_2 and configuration m . Consider the term $m_L := \text{let } x \leftarrow \{\text{let } y \leftarrow n_1 \text{ in } n_2\}!$ in m , which reduces to $m_R := \text{let } x \leftarrow (\text{let } y \leftarrow n_1 \text{ in } n_2)$ in m . The left-hand side translates to itself $m'_L := m_L$, since it's already in CCNF, while the right-hand side translates to $m'_R := \text{let } y \leftarrow n_1 \text{ in let } x \leftarrow n_2 \text{ in } m$. The goal is to show that m'_L reduces to m'_R . We must then show that $\text{let } x \leftarrow (\text{let } y \leftarrow n_1 \text{ in } n_2) \text{ in } m \rightsquigarrow^* \text{let } y \leftarrow n_1 \text{ in let } x \leftarrow n_2 \text{ in } m$, but there is no such reduction sequence since there is no reduction step that commutes let bindings. $\square$

However, if we know that n_1 in the counterexample reduces to some return v , then we can deduce that the right side and its translation must reduce to $\text{let } x \leftarrow n_2\{y \mapsto v\}$ in m , which gives us an equivalence between the original term and its translation. If our counterexample is well typed, then a logical equivalence gives us precisely the required information that subterms must reduce to canonical terms such as returned values.

To handle translations with arbitrary translation continuation K , we generalize to proving that a translation $\llbracket m \rrbracket K$ must be equivalent to plugging the computation back in as $K[m]$. The proof of Theorem 5.1 then proceeds by:

1. Defining a standard logical equivalence over CBPV types;
2. Closing over term and join point contexts with a semantic equivalence;
3. Proving the fundamental theorem of this equivalence, namely that well-typed terms are semantically equivalent to themselves;
4. Showing that well-typed commuting conversions are in the semantic equivalence;
5. Defining semantic equivalence of continuations and proving its fundamental theorem;
6. Showing that plugging continuations respects semantic equivalence; and finally,
7. Proving that given a well-typed computation and a well-typed continuation, plugging the continuation with the computation is equivalent to translating the computation using the continuation.

The theorem then holds by instantiating with the empty continuation. Section 5.1 covers step 1 to 3, Section 5.2 covers step 4, Section 5.3 covers step 5 to 6, and Section 5.4 covers step 7.

$$\begin{array}{l}
\boxed{(v, w) \in \llbracket A \rrbracket} \quad \boxed{(m, n) \in \llbracket B \rrbracket} \quad \boxed{(m, n) \in \llbracket B \rrbracket^*} \\
((), ()) \in \llbracket \top \rrbracket \\
(\text{inl } v, \text{inl } w) \in \llbracket A_1 + A_2 \rrbracket \text{ iff } (v, w) \in \llbracket A_1 \rrbracket \\
(\text{inr } v, \text{inr } w) \in \llbracket A_1 + A_2 \rrbracket \text{ iff } (v, w) \in \llbracket A_2 \rrbracket \\
(\{m\}, \{n\}) \in \llbracket \cup B \rrbracket \text{ iff } (m, n) \in \llbracket B \rrbracket^* \\
(\text{return } v, \text{return } w) \in \llbracket F A \rrbracket \text{ iff } (v, w) \in \llbracket A \rrbracket \\
(\lambda x. m, \lambda x. n) \in \llbracket A \rightarrow B \rrbracket \text{ iff } \forall v, w. (v, w) \in \llbracket A \rrbracket \Rightarrow (m\{x \mapsto v\}, n\{x \mapsto w\}) \in \llbracket B \rrbracket^* \\
(\langle m_1, m_2 \rangle, \langle n_1, n_2 \rangle) \in \llbracket B_1 \& B_2 \rrbracket \text{ iff } (m_1, n_1) \in \llbracket B_1 \rrbracket^* \wedge (m_2, n_2) \in \llbracket B_2 \rrbracket^* \\
(m, n) \in \llbracket B \rrbracket^* \text{ iff } \exists tm_1, tm_2. (m \Downarrow tm_1) \wedge (n \Downarrow tm_2) \wedge (tm_1, tm_2) \in \llbracket B \rrbracket
\end{array}$$

Fig. 12. Logical equivalence of terms over CBPV types [Equivalence: $\mathcal{V}, \mathcal{C}, \mathcal{E}$]

5.1 Logical Equivalence and the Fundamental Theorem

Our logical relation relates two closed values or computations at a value or computation type, respectively. We equivalently say that a pair of terms are in the interpretation of some type. Following [Forster et al. \[2019\]](#), we use an auxiliary interpretation $\llbracket B \rrbracket^*$ which relates computations that normalize to terminals related at B . It follows that if $(m, n) \in \llbracket B \rrbracket$, then $(m, n) \in \llbracket B \rrbracket^*$.

The interpretations, defined by mutual recursion over types, are otherwise standard: unit values are related, left and right injections are related when their values are related at the respective left or right type, thunks are related when their computations are related, functions are related when their bodies are related for all related arguments, and pairs are related when their first and second components are respectively related.

The first property we need of logical equivalence is backward closure under evaluation of $\llbracket B \rrbracket^*$, which holds by unfolding its definition and transitivity of evaluation. A helpful corollary adds weakening by join points; the join points themselves need not be equivalent, as they get dropped during evaluation.

LEMMA 5.2 (BACKWARD CLOSURE). [Equivalence: $\mathcal{E}$.bwds]

If $m_1 \rightsquigarrow^* m_2$ and $n_1 \rightsquigarrow^* n_2$ and $(m_2, n_2) \in \llbracket B \rrbracket^*$, then $(m_1, n_1) \in \llbracket B \rrbracket^*$.

LEMMA 5.3 (BACKWARD CLOSURE UNDER JOIN POINTS). [Equivalence: $\mathcal{E}$.bwdsRejoin]

If $m_1 \rightsquigarrow^* m_2$ and $n_1 \rightsquigarrow^* n_2$ and $(m_2, n_2) \in \llbracket B \rrbracket^*$, then $(\text{join } jx = m' \text{ in } m_1, \text{join } jx = n' \text{ in } n_1) \in \llbracket B \rrbracket^*$, using rules [E-JOIN](#) and [E-DROP](#).

The second property is symmetry and transitivity, making it a partial equivalence relation (PER) over all terms. These lemmas are proven by induction over the type and unfolding definitions.

LEMMA 5.4 (SYMMETRY AND TRANSITIVITY (LOGICAL EQUIVALENCE)).

- If $(v, w) \in \llbracket A \rrbracket$ then $(w, v) \in \llbracket A \rrbracket$. [Equivalence: $\mathcal{V}$.sym]
- If $(m, n) \in \llbracket B \rrbracket$ then $(n, m) \in \llbracket B \rrbracket$. [Equivalence: $\mathcal{C}$.sym]
- If $(m, n) \in \llbracket B \rrbracket^*$ then $(n, m) \in \llbracket B \rrbracket^*$. [Equivalence: $\mathcal{E}$.sym]
- If $(v_1, v_2) \in \llbracket A \rrbracket$ and $(v_2, v_3) \in \llbracket A \rrbracket$ then $(v_1, v_3) \in \llbracket A \rrbracket$. [Equivalence: $\mathcal{V}$.trans]
- If $(m_1, m_2) \in \llbracket B \rrbracket$ and $(m_2, m_3) \in \llbracket B \rrbracket$ then $(m_1, m_3) \in \llbracket B \rrbracket$. [Equivalence: $\mathcal{C}$.trans]
- If $(m_1, m_2) \in \llbracket B \rrbracket^*$ and $(m_2, m_3) \in \llbracket B \rrbracket^*$ then $(m_1, m_3) \in \llbracket B \rrbracket^*$. [Equivalence: $\mathcal{E}$.trans]

To work with equivalence of well-typed terms, which may be open with respect to value and jump contexts, we need to close over value and jump variables using substitution maps σ and join stacks φ , defined below. Applying a substitution map $v\{\sigma\}$, $m\{\sigma\}$ corresponds to simultaneous

$$\begin{array}{c}
\boxed{(\sigma, \tau) \in \llbracket \Gamma \rrbracket} \quad \boxed{(\varphi, \psi) \in \llbracket \Delta \rrbracket} \\
\\
\begin{array}{ccc}
\text{S-NIL} & \text{S-CONS} & \text{J-NIL} \\
\frac{}{(\cdot, \cdot) \in \cdot} & \frac{(\sigma, \tau) \in \llbracket \Gamma \rrbracket \quad (v, w) \in \llbracket A \rrbracket}{((\sigma, x \mapsto v), (\tau, x \mapsto w)) \in \llbracket \Gamma, x : A \rrbracket} & \frac{}{(\cdot, \cdot) \in \cdot} \\
\\
\text{J-CONS} \\
\frac{(\varphi, \psi) \in \llbracket \Delta \rrbracket \quad \forall v, w. (v, w) \in \llbracket A \rrbracket \Rightarrow (\text{joints } \varphi \text{ in } m\{x \mapsto v\}, \text{joints } \psi \text{ in } n\{x \mapsto w\}) \in \llbracket B \rrbracket^*}{((\varphi, j \ x = m), (\psi, j \ x = n)) \in \llbracket \Delta, j : A \nearrow B \rrbracket}
\end{array}
\end{array}$$

Fig. 13. Logical equivalence of substitution maps and join stacks [Equivalence:semCtxt, semDtxt]

substitution (so order doesn't matter in σ). Given a join stack $\varphi \equiv j_1 \ x_1 = m_1, \dots, j_i \ x_i = m_i$, the computation joins φ in m represents wrapping m in join points outward in, corresponding to the term $\text{join } j_1 \ x_1 = m_1 \text{ in } \dots \text{join } j_i \ x_i = m_i \text{ in } m$ (so order *does* matter in φ).

$$\sigma, \tau ::= \cdot \mid \sigma, x \mapsto v \quad \varphi, \psi ::= \cdot \mid \varphi, j \ x = m \quad [\text{Rejoin:J}]$$

We extend logical equivalence to substitution maps and join stacks, relating two of them at a particular context. Figure 13 gives inductive rules for these logical equivalences. Rules S-NIL and S-CONS are equivalent to stating that $(\sigma, \tau) \in \llbracket \Gamma \rrbracket$ holds when for all $x : A \in \Gamma$, $(\sigma(x), \tau(x)) \in \llbracket A \rrbracket$ holds. Rule J-CONS states that we can extend a pair of related join stacks φ and ψ by join points m and n of type $A \nearrow B$ when the join points themselves are related at B given arguments related at A and closed over by φ and ψ . They need to be closed over since m and n themselves may jump to earlier join points. We also need to track B in the jump context to later prove that jumps to related join points are themselves related at the same B .

We show that substitution maps and join stacks are PERs by induction on their derivations.

LEMMA 5.5 (SYMMETRY AND TRANSITIVITY (LOGICAL EQUIVALENCE AT CONTEXTS)).

- If $(\sigma, \tau) \in \llbracket \Gamma \rrbracket$ then $(\tau, \sigma) \in \llbracket \Gamma \rrbracket$. [Equivalence:semCtxt.sym]
- If $(\varphi, \psi) \in \llbracket \Delta \rrbracket$ then $(\psi, \varphi) \in \llbracket \Delta \rrbracket$. [Equivalence:semDtxt.sym]
- If $(\sigma_1, \sigma_2) \in \llbracket \Gamma \rrbracket$ and $(\sigma_2, \sigma_3) \in \llbracket \Gamma \rrbracket$ then $(\sigma_1, \sigma_3) \in \llbracket \Gamma \rrbracket$. [Equivalence:semCtxt.trans]
- If $(\varphi_1, \varphi_2) \in \llbracket \Delta \rrbracket$ and $(\varphi_2, \varphi_3) \in \llbracket \Delta \rrbracket$ then $(\varphi_1, \varphi_3) \in \llbracket \Delta \rrbracket$. [Equivalence:semDtxt.trans]

Using substitution maps and join stacks, we can define semantic equivalence of open terms. They are also partial equivalence relations, which follows from Lemmas 5.4 and 5.5. In all proofs that follow, we freely use symmetry and transitivity without explicit reference.

Definition 5.6 (Semantic equivalence of values). [Equivalence:semVal]

Values v and w are semantically equivalent under context Γ at type A , written $\Gamma \models v \sim w : A$, when for all substitution maps $(\sigma, \tau) \in \llbracket \Gamma \rrbracket$, $(v\{\sigma\}, w\{\tau\}) \in \llbracket A \rrbracket$ holds.

Definition 5.7 (Semantic equivalence of computations). [Equivalence:semCom]

Computations m and n are semantically equivalent under contexts Γ and Δ at type B , written $\Gamma \mid \Delta \models m \sim n : B$, when for all substitution maps $(\sigma, \tau) \in \llbracket \Gamma \rrbracket$ and join stacks $(\varphi, \psi) \in \llbracket \Delta \rrbracket$, $(\text{joints } \varphi \text{ in } m\{\sigma\}, \text{joints } \psi \text{ in } n\{\tau\}) \in \llbracket B \rrbracket^*$ holds.

LEMMA 5.8 (SYMMETRY AND TRANSITIVITY (SEMANTIC EQUIVALENCE)).

- If $\Gamma \models v \sim w : A$ then $\Gamma \models w \sim v : A$. [Equivalence:semVal.sym]
- If $\Gamma \mid \Delta \models m \sim n : B$ then $\Gamma \mid \Delta \models n \sim m : B$. [Equivalence:semCom.sym]
- If $\Gamma \models v_1 \sim v_2 : A$ and $\Gamma \models v_2 \sim v_3 : A$ then $\Gamma \models v_1 \sim v_3 : A$. [Equivalence:semVal.trans]

- If $\Gamma \mid \Delta \models m_1 \sim m_2 : B$ and $\Gamma \mid \Delta \models m_2 \sim m_3 : B$ then $\Gamma \mid \Delta \models m_1 \sim m_3 : B$.
[Equivalence:semCom.trans]

Finally, we show the fundamental theorem of semantic equivalence: well-typed terms are semantically equivalent to themselves.

THEOREM 5.9 (FUNDAMENTAL THEOREM OF SEMANTIC EQUIVALENCE). [Equivalence:soundness] *If $\Gamma \vdash v : A$ then $\Gamma \models v \sim v : A$, and if $\Gamma \mid \Delta \vdash m : B$ then $\Gamma \mid \Delta \models m \sim m : B$.*

PROOF. By induction on the typing derivations of v and m . Letting $(\sigma, \tau) \in \llbracket \Gamma \rrbracket$ and $(\varphi, \psi) \in \llbracket \Delta \rrbracket$, the goal is to show that $(v\{\sigma\}, v\{\tau\}) \in \llbracket A \rrbracket$ and $(\text{joins } \varphi \text{ in } m\{\sigma\}, \text{joins } \psi \text{ in } m\{\tau\}) \in \llbracket B \rrbracket^*$ hold.

- **Case T-VAR:** Holds by $(\sigma, \tau) \in \llbracket \Gamma \rrbracket$ on the variable.
- **Cases T-UNIT, T-LEFT, T-RIGHT, and T-THUNK:** Hold by definition of the logical equivalence, using the induction hypotheses on any subterms.
- **Cases T-FUN, T-RET, and T-PROD:** Hold by Lemma 5.3 with no reduction, using the induction hypotheses on subterms.
- **Cases T-APP, T-FST, and T-SND:** By the induction hypotheses on the scrutinee premises, they must reduce to related functions, related first pair projections, or related second pair projections, respectively. Then the goal holds by Lemma 5.3 on these relations, reducing by E-APP, E-FST, or E-SND, respectively.
- **Cases T-LET and T-CASE:** By the induction hypotheses on the scrutinee premises, they must reduce to related returns, related left injections, or related right injections, respectively. Then the goal holds by Lemma 5.2 on these relations, reducing by E-RET, E-LEFT, or E-RIGHT, respectively.
- **Case T-JOIN:** Holds by the induction hypothesis on the join expression body, extending semantic equivalence of join stacks using the induction hypothesis on the join points.
- **Case T-JUMP:** Holds by $(\varphi, \psi) \in \llbracket \Delta \rrbracket$ on the jump variable, instantiating the join point by the induction hypothesis on related values. $\square$

As a corollary, we have type safety and normalization: a closed, well-typed computation never evaluates to a stuck term, and moreover evaluates to a terminal.

COROLLARY 5.10 (NORMALIZATION). [Equivalence:safety] *If $\cdot \mid \cdot \vdash m : B$ then $\exists tm, m \rightsquigarrow^* tm$.*

5.2 Semantic Equivalence of Commuting Conversions

As the translation normalizes with respect to all commuting conversions, to show that semantic equivalence of the translation, naturally we need to show semantic equivalence of the commuting conversions. Recall that with four evaluation contexts and two tail contexts, there are eight total commuting conversions to handle, each assuming well-typedness of various subterms. We present them as derivable rules in Figure 14 for legibility.

The general strategy to proving these is by using the Theorem 5.9 on the typing derivations, extracting a logical equivalence for the desired type, applying Lemma 5.2 or Lemma 5.3, then using the evaluation rules and Corollary 3.4 to find the correct evaluation sequence from the left and right sides of the lemma statement to the respective sides of the extracted logical equivalence. We step through only the proof of LET-APP as a representative case.

PROOF. Because substitution maps and join stacks don't play much of a role, we mostly ignore them here. The goal is then to show that $(\text{let } x \leftarrow n \text{ in } m) v$ is equivalent to $\text{let } x \leftarrow n \text{ in } (m v)$, knowing that all subterms are well typed. By Theorem 5.9, we have that n reduces to some return w , and that $m\{x \mapsto w\}$ reduces to some $\lambda y. m'$. Then we know that the right-hand side reduces by E-RET and E-APP to $m'\{y \mapsto v\}$. We also have that $\text{let } x \leftarrow n \text{ in } m$ reduces to $m\{x \mapsto w\}$ by E-RET.

$$\begin{array}{c}
\frac{\Gamma \mid \cdot \vdash \text{let } x \leftarrow n \text{ in } m : F A \quad \Gamma, y : A \mid \Delta \vdash m' : B}{\Gamma \mid \Delta \models \text{let } y \leftarrow (\text{let } x \leftarrow n \text{ in } m) \text{ in } m' \sim \text{let } x \leftarrow n \text{ in let } y \leftarrow m \text{ in } m' : B} \text{LET-LET} \\
\\
\frac{\Gamma \mid \cdot \vdash \text{let } x \leftarrow n \text{ in } m : A \rightarrow B \quad \Gamma \vdash v : A}{\Gamma \mid \Delta \models (\text{let } x \leftarrow n \text{ in } m) v \sim \text{let } x \leftarrow n \text{ in } (m v) : B} \text{LET-APP} \\
\\
\frac{\Gamma \mid \cdot \vdash \text{let } x \leftarrow n \text{ in } m : B_1 \& B_2}{\Gamma \mid \Delta \models \text{fst} (\text{let } x \leftarrow n \text{ in } m) \sim \text{let } x \leftarrow n \text{ in } (\text{fst } m) : B_1} \text{LET-FST} \\
\\
\frac{\Gamma \mid \cdot \vdash \text{let } x \leftarrow n \text{ in } m : B_1 \& B_2}{\Gamma \mid \Delta \models \text{snd} (\text{let } x \leftarrow n \text{ in } m) \sim \text{let } x \leftarrow n \text{ in } (\text{snd } m) : B_2} \text{LET-SND} \\
\\
\frac{\Gamma \mid \cdot \vdash \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} : F A \quad \Gamma, z : A \mid \Delta \vdash m : B}{\Gamma \mid \Delta \models \text{let } z \leftarrow (\text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\}) \text{ in } m \sim \text{case } v \text{ of } \{\text{inl } x \Rightarrow (\text{let } z \leftarrow m_1 \text{ in } m); \text{inr } y \Rightarrow (\text{let } z \leftarrow m_2 \text{ in } m)\} : B} \text{CASE-LET} \\
\\
\frac{\Gamma \mid \cdot \vdash \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} : A \rightarrow B \quad \Gamma \vdash w : A}{\Gamma \mid \Delta \models (\text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\}) w \sim \text{case } v \text{ of } \{\text{inl } x \Rightarrow (m_1 w); \text{inr } y \Rightarrow (m_2 w)\} : B} \text{CASE-APP} \\
\\
\frac{\Gamma \mid \cdot \vdash \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} : B_1 \& B_2}{\Gamma \mid \Delta \models \text{fst} (\text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\}) \sim \text{case } v \text{ of } \{\text{inl } x \Rightarrow (\text{fst } m_1); \text{inr } y \Rightarrow (\text{fst } m_2)\} : B_1} \text{CASE-FST} \\
\\
\frac{\Gamma \mid \cdot \vdash \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} : B_1 \& B_2}{\Gamma \mid \Delta \models \text{snd} (\text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\}) \sim \text{case } v \text{ of } \{\text{inl } x \Rightarrow (\text{snd } m_1); \text{inr } y \Rightarrow (\text{snd } m_2)\} : B_2} \text{CASE-SND}
\end{array}$$

Fig. 14. Semantic equivalence of commuting conversions

[Commutation: letLet, appLet, fstLet, sndLet, letCase, appCase, fstCase, sndCase]

Then we know that the left-hand side reduces by **E-APP** also to $m'\{y \mapsto v\}$. Therefore, by **Lemma 5.3**, the left- and right-hand sides are equivalent. $\square$

Although our translation doesn't involve commuting join points, we do require a corresponding semantic equivalence to unnest join points, stated in **Figure 15**.

JOIN-JOIN

$$\frac{\Gamma, x_1 : A_1 \mid \Delta \vdash m_1 : B \quad \Gamma, x_2 : A_2 \mid \Delta, j_1 : A_1 \nearrow B \vdash m_2 : B \quad \Gamma \mid \Delta, j_2 : A_2 \nearrow B \vdash m : B}{\Gamma \mid \Delta \models \text{join } j_2 \ x_2 = (\text{join } j_1 \ x_1 = m_1 \text{ in } m_2) \text{ in } m \sim \text{join } j_1 \ x_1 = m_1 \text{ in join } j_2 \ x_2 = m_2 \text{ in } m : B}$$

Fig. 15. Semantic equivalence of commuting join points [Commutation: joinJoin]

Only the join point j_1 is in scope in m_2 , while only the join point j_2 is in scope in m . If m jumps to j_2 , then both sides reduce to the same computation. If m doesn't jump to j_2 , then both join points can be dropped. In the mechanization, this isn't so easy to prove, since we're using de Bruijn

indexed jump variables. Consequently, m must be explicitly weakened on the right-hand side, and to show that join points can be dropped, we first need to show that evaluation preserves weakening. These technical details aside, the proof proceeds similarly to those for the commuting conversions.

5.3 Semantic Equivalence of Plugging Continuations

Just as the [Fundamental theorem of semantic equivalence](#) requires related substitution maps and related join stacks, for the final proof in the next section, we need a notion of related continuations. Rather than first defining a logical equivalence over continuations, we directly define semantic equivalence in a way that incorporates plugging. Here, $K\{\sigma\}$ denotes applying the substitution map σ to all value and computation subterms of K .

Definition 5.11 (Semantic equivalence of continuations). [\[Soundness:semK\]](#)

Continuations K_1 and K_2 are semantically equivalent under Γ and Δ at type $B_1 \Rightarrow B_2$, written $\boxed{\Gamma \mid \Delta \models K_1 \sim K_2 : B_1 \Rightarrow B_2}$, when for all substitution maps $(\sigma, \tau) \in \llbracket \Gamma \rrbracket$, join stacks $(\varphi, \psi) \in \llbracket \Delta \rrbracket$, and computations $(n_1, n_2) \in \llbracket B_1 \rrbracket^*$, (joins φ in $K\{\sigma\}[n_1]$, joins ψ in $K\{\tau\}[n_2]$) $\in \llbracket B_2 \rrbracket$ holds.

A fundamental theorem for semantic equivalence of continuations holds as well, which relies on the corresponding theorem for terms, as well as congruence of evaluating under plugging.

LEMMA 5.12 (E-PLUG). [\[CCNF:Evals.plug\]](#)

If $n_1 \rightsquigarrow^* n_2$, then $K[n_1] \rightsquigarrow^* K[n_2]$, by induction on the structure of K .

THEOREM 5.13 (FUNDAMENTAL THEOREM OF SEMANTIC EQUIVALENCE OF CONTINUATIONS).

If $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$ then $\Gamma \mid \Delta \models K \sim K : B_1 \Rightarrow B_2$. [\[Soundness:soundK\]](#)

PROOF. By induction on the typing derivation of K . In the **K-APP** case, use [Theorem 5.9](#) on the value argument, and in the **K-LET** case, use [Theorem 5.9](#) on the computation body. In the **K-APP**, **K-FST**, and **K-SND** cases, use **E-PLUG** to reduce under the rest of the continuation. $\square$

Plugging semantically equivalent computations into both the same continuation and equivalent continuations then yields equivalent computations as long as substitution commutes with plugging.

LEMMA 5.14 (SUBSTITUTION COMMUTES WITH PLUGGING). [\[CCNF:substPlug\]](#)

$(K[n])\{\sigma\} = (K\{\sigma\})[n\{\sigma\}]$, by induction on the structure of K .

LEMMA 5.15 (SEMANTIC EQUIVALENCE OF PLUGGING). [\[Soundness:semK.plug\]](#)

If $\Gamma \mid \Delta \models K_1 \sim K_2 : B_1 \Rightarrow B_2$ and $\Gamma \mid \cdot \models n_1 \sim n_2 : B_1$, then $\Gamma \mid \Delta \models K_1[n_1] \sim K_2[n_2] : B_2$, trivially after rewriting by [Lemma 5.14](#).

COROLLARY 5.16. [\[Soundness:semPlug\]](#) If $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$ and $\Gamma \mid \cdot \models n_1 \sim n_2 : B_1$, then

$\Gamma \mid \Delta \models K[n_1] \sim K[n_2] : B_2$, using [Lemma 5.15](#) and [Theorem 5.13](#).

5.4 Commuting Conversion Normalization is Semantically Equivalent to Plugging

Now that we have both semantic equivalence of commuting conversions and of plugging, we use them together to show that plugging commutes with tail contexts.

LEMMA 5.17 (SEMANTIC EQUIVALENCE OF PLUGGING LET EXPRESSIONS). [\[Soundness:semKletin\]](#)

If $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$ and $\Gamma \mid \cdot \vdash \text{let } x \leftarrow n \text{ in } m : B_1$, then $\Gamma \mid \Delta \models K[\text{let } x \leftarrow n \text{ in } m] \sim \text{let } x \leftarrow n \text{ in } K[m] : B_2$.

PROOF. By induction on the typing derivation of K . In the **K-NIL** case, this holds by [Theorem 5.9](#). In all other cases, the left-hand side involves a subterm of the form $\text{let } y \leftarrow (\text{let } x \leftarrow n \text{ in } m) \text{ in } m'$, or $(\text{let } x \leftarrow n \text{ in } m) \text{ v}$, or $\text{fst}(\text{let } x \leftarrow n \text{ in } m)$, or $\text{snd}(\text{let } x \leftarrow n \text{ in } m)$, so we use derived

rules **LET-LET**, **LET-APP**, **LET-FST**, and **LET-SND** respectively to commute them. For case **K-LET**, we are done; for the remaining cases, we require [Corollary 5.16](#) to commute under a plugged K , then use the induction hypothesis to transitively connect to the right-hand side. $\square$

LEMMA 5.18 (SEMANTIC EQUIVALENCE OF PLUGGING CASE EXPRESSIONS). [[Soundness:semKcase](#)]
*If $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$ and $\Gamma \mid \cdot \vdash \text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} : B_1$,
 then $\Gamma \mid \Delta \models K[\text{case } v \text{ of } \{\text{inl } x \Rightarrow m_1; \text{inr } y \Rightarrow m_2\}] \sim \text{case } v \text{ of } \{\text{inl } x \Rightarrow K[m_1]; \text{inr } y \Rightarrow K[m_2]\} : B_2$.*

PROOF. By induction on the typing derivation of K , using [Theorem 5.9](#) and [Corollary 5.16](#) along with derived rules **CASE-LET**, **CASE-APP**, **CASE-FST**, and **CASE-SND** similarly to [Lemma 5.17](#). $\square$

These two lemmas suffice to prove the desired equivalence for a naïve translation that duplicates continuations instead of using join points. To handle join points, we need one more lemma that gives an equivalence between a translation that doesn't use a join point and one that does.

LEMMA 5.19 (JUMP EQUIVALENCE OF PLUGGING). [[Soundness:semKjoin](#)]
*If $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$, $\Gamma \mid \cdot \vdash n : B_1$, and $K \equiv (\text{let } x \leftarrow \square \text{ in } m) :: \dots :: k_i$, then
 $\Gamma \mid \Delta \models K[n] \sim \text{join } j \ x = m \text{ in } K'[n] : B_2$, where $K' \equiv (\text{let } x \leftarrow \square \text{ in jump } j \ x) :: \dots :: k_i$.*

PROOF. By induction on the typing derivation of K . The rule **K-LET** case uses [Theorem 5.9](#) on the typing derivation of n to show it must evaluate to some return v ; then both sides evaluate to $m\{x \mapsto v\}$, and we get the equivalence from [Theorem 5.9](#) on the typing derivation of m . The remaining cases follow from the induction hypotheses. $\square$

LEMMA 5.20 (JUMP EQUIVALENCE OF THE TRANSLATION). [[Soundness:soundCCjoin](#)]
*Suppose that m contains no join points or jumps. If $\Gamma \mid \Delta \vdash K : B_1 \Rightarrow B_2$, $\Gamma \mid \Delta \vdash m : B_1$, and
 $K \equiv (\text{let } x \leftarrow \square \text{ in } m') :: \dots :: k_i$, then $\Gamma \mid \Delta \models \llbracket m \rrbracket K \sim \text{join } j \ x = m' \text{ in } \llbracket m \rrbracket K' : B_2$, where
 $K' \equiv (\text{let } x \leftarrow \square \text{ in jump } j \ x) :: \dots :: k_i$.*

PROOF. By induction on the typing derivation of m .

- **Cases T-FORCE, T-FUN, T-RET, and T-PROD:** Hold directly from [Lemma 5.19](#), using [Lemma 4.5](#) to type translated subterms.
- **Cases T-APP, T-FST, and T-SND:** Hold directly from the induction hypotheses.
- **Case T-LET:** The goal is to show that $\llbracket \text{let } y \leftarrow n \text{ in } m \rrbracket K$ is semantically equivalent to $\text{join } j \ x = m' \text{ in } \llbracket \text{let } y \leftarrow n \text{ in } m \rrbracket K'$, which holds by the following chain of equivalences.

$$\begin{aligned}
 & \llbracket \text{let } y \leftarrow n \text{ in } m \rrbracket K \\
 &= \llbracket n \rrbracket (\text{let } y \leftarrow \square \text{ in } \llbracket m \rrbracket K) && \text{(by definition)} \\
 &\sim \text{join } j' \ y = \llbracket m \rrbracket K \text{ in } \llbracket n \rrbracket (\text{let } y \leftarrow \square \text{ in jump } j' \ y) && \text{(by IH on } n) \\
 &\sim \text{join } j' \ y = (\text{join } j \ x = m' \text{ in } \llbracket m \rrbracket K') \text{ in } \llbracket n \rrbracket (\text{let } y \leftarrow \square \text{ in jump } j' \ y) && \text{(by IH on } m) \\
 &\sim \text{join } j \ x = m' \text{ in join } j' \ y = \llbracket m \rrbracket K' \text{ in } \llbracket n \rrbracket (\text{let } y \leftarrow \square \text{ in jump } j' \ y) && \text{(by JOIN-JOIN)} \\
 &\sim \text{join } j \ x = m' \text{ in } \llbracket n \rrbracket (\text{let } y \leftarrow \square \text{ in } \llbracket m \rrbracket K') && \text{(by IH on } n) \\
 &= \text{join } j \ x = m' \text{ in } \llbracket \text{let } y \leftarrow n \text{ in } m \rrbracket K' && \text{(by definition)}
 \end{aligned}$$

- **Case T-CASE:** Let the translated computation be $\text{case } v \text{ of } \{\text{inl } y \Rightarrow m_1; \text{inr } y \Rightarrow m_2\}$. There are two subcases depending on what m' is.

– If $m' \equiv \text{jump } j' \ w$, then by the translation, the left-hand side is

$$\text{case } \llbracket v \rrbracket \text{ of } \{\text{inl } y \Rightarrow \llbracket m_1 \rrbracket K; \text{inr } y \Rightarrow \llbracket m_2 \rrbracket K\},$$

and the right-hand side is

$$\text{join } j \ x = m' \text{ in case } \llbracket v \rrbracket \text{ of } \{\text{inl } y \Rightarrow \llbracket m_1 \rrbracket K'; \text{inr } y \Rightarrow \llbracket m_2 \rrbracket K'\}.$$

By [Theorem 5.9](#), we know $\llbracket v \rrbracket$ reduces to either some $\text{inl } w$ or $\text{inr } w$. WLOG, supposing the former, it then suffices to show that $\llbracket m_1 \rrbracket K \{y \mapsto w\}$ is semantically equivalent to $\text{join } j \ x = m' \text{ in } \llbracket m_1 \rrbracket K' \{y \mapsto w\}$, which holds by the induction hypothesis on m_1 .

– Otherwise, both the left- and right-hand sides are

$$\text{join } j \ x = m' \text{ in case } \llbracket v \rrbracket \text{ of } \{\text{inl } y \Rightarrow \llbracket m_1 \rrbracket K'; \text{inr } y \Rightarrow \llbracket m_2 \rrbracket K'\},$$

which are equivalent by [Theorem 5.9](#), using [Lemma 4.5](#) for the translated subterms. $\square$

Finally, we have all pieces to prove the main theorem: the translation under continuation K is semantically equivalent to plugging into K . The equivalence we want follows from instantiating by the empty continuation.

THEOREM 5.21. [[Soundness](#):soundCC] *Suppose that v and m are terms containing no join points and jumps, and K_1, K_2 are continuations that may contain join points and jumps.*

- If $\Gamma \vdash v : A$ then $\Gamma \models v \sim \llbracket v \rrbracket : A$.
- If $\Gamma \mid \cdot \vdash m : B_1, \Gamma \mid \Delta \vdash K_1 : B_1 \Rightarrow B_2, \Gamma \mid \Delta \vdash K_2 : B_1 \Rightarrow B_2$, and $\Gamma \mid \Delta \models K_1 \sim K_2 : B_1 \Rightarrow B_2$, then $\Gamma \mid \Delta \models K_1[m] \sim \llbracket m \rrbracket K_2 : B_2$.

PROOF. By mutual induction on the typing derivations of v and m .

- **Case T-VAR:** Holds by logical equivalence of substitution maps.
- **Case T-UNIT:** Trivial.
- **Cases T-FORCE, T-FUN, T-RET, and T-PROD:** Hold by [Lemma 5.15](#) and the induction hypotheses.
- **Cases T-LEFT, T-RIGHT, T-THUNK, T-APP, T-FST, and T-SND:** Hold by the induction hypotheses.
- **Case T-LET:** The goal is to show that $K[\text{let } x \leftarrow n \text{ in } m]$ is equivalent to $\llbracket \text{let } x \leftarrow n \text{ in } m \rrbracket K$, which holds by the following chain of equivalences.

$$\begin{aligned} K[\text{let } x \leftarrow n \text{ in } m] &\sim \text{let } x \leftarrow n \text{ in } K[m] && \text{(by Lemma 5.17)} \\ &\sim \text{let } x \leftarrow n \text{ in } \llbracket m \rrbracket K = (\text{let } x \leftarrow \square \text{ in } \llbracket m \rrbracket K)[n] && \text{(by IH on } m) \\ &\sim \llbracket n \rrbracket (\text{let } x \leftarrow \square \text{ in } \llbracket m \rrbracket K) = \llbracket \text{let } x \leftarrow n \text{ in } m \rrbracket K && \text{(by IH on } n) \end{aligned}$$

- **Case T-CASE:** The goal is to show that $K[\text{case } v \text{ of } \{\text{inl } y \Rightarrow m_1; \text{inr } y \Rightarrow m_2\}]$ is equivalent to $\llbracket \text{case } v \text{ of } \{\text{inl } y \Rightarrow m_1; \text{inr } y \Rightarrow m_2\} \rrbracket K$. By [Lemma 5.18](#), the left-hand side is equivalent to $\text{case } v \text{ of } \{\text{inl } y \Rightarrow K[m_1]; \text{inr } y \Rightarrow K[m_2]\}$. Furthermore, by the induction hypotheses on the subterms, this is equivalent to $\text{case } \llbracket v \rrbracket \text{ of } \{\text{inl } y \Rightarrow \llbracket m_1 \rrbracket K; \text{inr } y \Rightarrow \llbracket m_2 \rrbracket K\}$. It then suffices to prove this equivalent to the right-hand side. There are two subcases depending on what K is.
 - If $K \equiv \square :: \dots :: k_i$ or $K \equiv (\text{let } x \leftarrow \square \text{ in jump } j \ v) :: \dots :: k_i$, then the right-hand side is $\text{case } \llbracket v \rrbracket \text{ of } \{\text{inl } y \Rightarrow \llbracket m_1 \rrbracket K; \text{inr } y \Rightarrow \llbracket m_2 \rrbracket K\}$, and we are done.
 - Otherwise, $K \equiv (\text{let } x \leftarrow \square \text{ in } m') :: \dots :: k_i$. Letting $K' \equiv (\text{let } x \leftarrow \square \text{ in jump } j \ x) :: \dots :: k_i$, the right-hand side is

$$\text{join } j \ x = m' \text{ in case } \llbracket v \rrbracket \text{ of } \{\text{inl } y \Rightarrow \llbracket m_1 \rrbracket K'; \text{inr } y \Rightarrow \llbracket m_2 \rrbracket K'\}.$$

By [Theorem 5.9](#), we know $\llbracket v \rrbracket$ reduces to either some $\text{inl } w$ or $\text{inr } w$. WLOG, supposing the former, it then suffices to show that $\llbracket m_1 \rrbracket K \{y \mapsto w\}$ is equivalent to $\text{join } j \ x = m' \text{ in } \llbracket m_1 \rrbracket K' \{y \mapsto w\}$. Extending the substitution map with $y \mapsto w$ (noting that y is only free in m_1), this equivalence holds by [Lemma 5.20](#). $\square$

COROLLARY 5.22. [[Soundness](#):soundCCnil] *Suppose m contains no join points and jumps.*

If $\Gamma \mid \cdot \vdash m : B$ then $\Gamma \mid \cdot \models m \sim \llbracket m \rrbracket \square : B$.

At last we are able to prove [Theorem 5.1](#), restated below, with one more minor lemma.

LEMMA 5.23 (EQUIVALENT GROUND VALUES ARE EQUAL). [[Soundness](#): $\mathcal{V}$.ground]

If $(v, w) \in \llbracket T \rrbracket$ then $v = w$, proven by induction on T .

THEOREM 5.24. [Soundness:retGround]

Given m such that $\cdot \mid \cdot \vdash m : F T$, if $m \Downarrow \text{return } v$, then $\llbracket m \rrbracket \square \Downarrow \text{return } v$.

PROOF. From Corollary 5.22, we have $\cdot \mid \cdot \models m \sim \llbracket m \rrbracket \square : F T$. Instantiating with empty substitution maps and empty join stacks, we have $(m, \llbracket m \rrbracket \square) \in \llbracket F T \rrbracket^*$. By inversion, we know that there exists some $(w_1, w_2) \in \llbracket T \rrbracket$ such that $m \Downarrow \text{return } w_1$ and $\llbracket m \rrbracket \square \Downarrow \text{return } w_2$. By Corollary 3.3, we have $v = w_1$, and by Lemma 5.23, we have $w_1 = w_2$. Therefore, $\llbracket m \rrbracket \square \Downarrow \text{return } v$. $\square$

6 Discussion

6.1 Case-of-Case

In the lambda calculus with conditional expressions, the *case-of-case* transformation commutes a conditional whose scrutinee is another conditional. The naïve transformation directly moves the outer conditional into the branches of the inner, duplicating the code in its branches.

$$\begin{aligned} & \text{if (if } e_1 \text{ then } e_2 \text{ else } e_3) \text{ then } e_4 \text{ else } e_5 \\ & \implies \text{if } e_1 \text{ then (if } e_2 \text{ then } e_4 \text{ else } e_5) \text{ else (if } e_3 \text{ then } e_4 \text{ else } e_5) \end{aligned} \quad [9]$$

Let-binding the outer branches to $x = e_4$ and $y = e_5$ avoids this duplication. Notably, doing so doesn't touch the direct scrutinization of e_2 and e_3 , so if they are literal true or false values, then those branches can be inlined to just x or y .

$$\begin{aligned} & \text{if (if } e_1 \text{ then } e_2 \text{ else } e_3) \text{ then } e_4 \text{ else } e_5 \\ & \implies \text{let } x = e_4 \text{ in let } y = e_5 \text{ in if } e_1 \text{ then (if } e_2 \text{ then } x \text{ else } y) \text{ else (if } e_3 \text{ then } x \text{ else } y) \end{aligned} \quad [10]$$

If we translate the left-hand lambda expression to a CBPV term, both CBV and CBN translations yield a term of the following shape.

$$\text{let } y \leftarrow (\text{let } x \leftarrow m_1 \text{ in if } x \text{ then } m_2 \text{ else } m_3) \text{ in if } y \text{ then } m_4 \text{ else } m_5 \quad [11]$$

CC-normalizing this term moves the outer let binding into the branches of the conditional on x , with jumps to avoid duplication.

$$\begin{aligned} & \text{let } x \leftarrow m_1 \text{ in join } j \text{ } y = \text{if } y \text{ then } m_4 \text{ else } m_5 \text{ in} \\ & \text{if } x \text{ then (let } y \leftarrow m_2 \text{ in jump } j \text{ } y) \text{ else (let } y \leftarrow m_3 \text{ in jump } j \text{ } y) \end{aligned} \quad [12]$$

Unfortunately, even if m_2 is the computation return true, inlining it will only reduce the branch down to jump j true, whose evaluation will still need to jump to the conditional in the join point that could otherwise have been inlined away statically. What we would like is to create separate join points for m_4 and m_5 and jump to them individually to permit the kind of inlining above, similar to the right-hand side of Equation 10.

$$\begin{aligned} & \text{let } x \leftarrow m_1 \text{ in join } j_4 = m_4 \text{ in join } j_5 = m_5 \text{ in} \\ & \text{if } x \text{ then (let } y \leftarrow m_2 \text{ in if } y \text{ then jump } j_4 \text{ else jump } j_5) \\ & \quad \text{else (let } y \leftarrow m_3 \text{ in if } y \text{ then jump } j_4 \text{ else jump } j_5) \end{aligned} \quad [13]$$

Therefore, we prove a semantic equivalence between Equation 12 and Equation 13, generalizing to case expressions. Consequently, a transformation from the former to the latter is valid, and we recover the desired case-of-case transformation.

LEMMA 6.1 (CASE-OF-CASE). [Commutation:caseCase] Suppose the following hold:

- $\Gamma \vdash v : A_3 + A_4$
- $\Gamma, y_1 : A_1 \mid \Delta \vdash m_1 : B$
- $\Gamma, y_2 : A_2 \mid \Delta \vdash m_2 : B$
- $\Gamma, y_3 : A_3 \mid \cdot \vdash m_3 : F(A_1 + A_2)$

$$\bullet \Gamma, y_4 : A_4 \mid \cdot \vdash m_4 : F(A_1 + A_2)$$

Then under contexts $\Gamma \mid \Delta$, the computation

$$\begin{aligned} & \text{join } j \ x = \text{case } x \text{ of } \{\text{inl } y_1 \Rightarrow m_1; \text{inr } y_2 \Rightarrow m_2\} \text{ in} \\ & \quad \text{case } v \text{ of } \{\text{inl } y_3 \Rightarrow (\text{let } x \leftarrow m_3 \text{ in jump } j \ x); \text{inr } y_4 \Rightarrow (\text{let } x \leftarrow m_4 \text{ in jump } j \ x)\} \end{aligned}$$

is semantically equivalent at type B to the computation

$$\begin{aligned} & \text{join } j_1 \ y_1 = m_1 \text{ in join } j_2 \ y_2 = m_2 \text{ in} \\ & \quad \text{case } v \text{ of } \{\text{inl } y_3 \Rightarrow (\text{let } x \leftarrow m_3 \text{ in case } x \text{ of } \{\text{inl } y_1 \Rightarrow \text{jump } j_1 \ y_1; \text{inr } y_2 \Rightarrow \text{jump } j_2 \ y_2\}); \\ & \quad \text{inr } y_4 \Rightarrow (\text{let } x \leftarrow m_4 \text{ in case } x \text{ of } \{\text{inl } y_1 \Rightarrow \text{jump } j_1 \ y_1; \text{inr } y_2 \Rightarrow \text{jump } j_2 \ y_2\})\} \end{aligned}$$

This transformation applies generally to case-of-case-of-case and so forth. The translation to CBPV gives join points that contain join points that contain case expressions and so forth. Unnesting these join points with **JOIN-JOIN** followed by [Lemma 6.1](#) yields the desired transformation.

6.2 Inlining and CCNF

Recall that new opportunities for inlining revealed by CC-normalization preserve CCNF, since they only occur in tail positions. However, subsequent inlinings that force direct thunks may violate CCNF. Consider the following sequence of a commuting conversion, followed by an inlining that reduces a function, followed by an inlining that forces a thunk.

$$\begin{aligned} & (\text{let } x \leftarrow n_1 \text{ in } (\lambda y'. \text{let } y \leftarrow y'! \text{ in } m_1)) \{\text{let } z \leftarrow n_2 \text{ in } m_2\} \\ & \quad \Rightarrow \text{let } x \leftarrow n_1 \text{ in } ((\lambda y'. \text{let } y \leftarrow y'! \text{ in } m_1) \{\text{let } z \leftarrow n_2 \text{ in } m_2\}) \quad (\text{commute}) \\ & \quad \Rightarrow \text{let } x \leftarrow n_1 \text{ in } (\text{let } y \leftarrow \{\text{let } z \leftarrow n_2 \text{ in } m_2\}! \text{ in } m_1) \quad (\text{E-APP}) \\ & \quad \Rightarrow \text{let } x \leftarrow n_1 \text{ in } (\text{let } y \leftarrow (\text{let } z \leftarrow n_2 \text{ in } m_2) \text{ in } m_1) \quad (\text{E-FORCE}) \end{aligned}$$

The resulting term after commuting and inlining once is still in CCNF. But if we force the thunk, we end up with a nested let expression, which isn't in CCNF. Therefore, renormalization may be required only after forcing thunks that appear in n positions as a result of inlining.

6.3 Commuting Constructors

Our notion of commuting conversions only involves elimination forms, as they make up evaluation contexts E and tail contexts L . Other work [[Forster et al. 2019](#); [Levy 2003](#)] also consider equations which commuting elimination forms with introduction forms, specifically computation constructors in tail position, listed below. In particular, the first equation is part of [Levy's sequencing laws](#).

$$\begin{aligned} & \text{let } x \leftarrow n \text{ in } \lambda y. m \iff \lambda y. \text{let } x \leftarrow n \text{ in } m \\ & \text{case } v \text{ of } \{\text{inl } y \Rightarrow \lambda x. m_1; \text{inr } z \Rightarrow \lambda x. m_2\} \iff \lambda x. \text{case } v \text{ of } \{\text{inl } y \Rightarrow m_1; \text{inr } z \Rightarrow m_2\} \\ & \text{let } x \leftarrow n \text{ in } \langle m_1, m_2 \rangle \iff \langle \text{let } x \leftarrow n \text{ in } m_1, \text{let } x \leftarrow n \text{ in } m_2 \rangle \\ & \text{case } v \text{ of } \{\text{inl } y \Rightarrow \langle m_1, m_2 \rangle; \text{inr } z \Rightarrow \langle m_3, m_4 \rangle\} \iff \\ & \quad \langle \text{case } v \text{ of } \{\text{inl } y \Rightarrow m_1; \text{inr } z \Rightarrow m_3\}, \text{case } v \text{ of } \{\text{inl } y \Rightarrow m_2; \text{inr } z \Rightarrow m_4\} \rangle \end{aligned}$$

We don't consider these because it's unclear which direction to pick and what benefits they provide. Going left to right, the case equations only apply when the branches are both exactly an introduction form, and these naïve pair equations duplicate code, so typing and evaluation would need to be extended to permit creating join points from the components and jumping from within pairs. Going right to left, the pair equations only apply when the projections both eliminate exactly the same thing. Either way, they are sensitive to slight syntactic variation, e.g. permuted let bindings. Furthermore, neither direction appears to reveal optimization opportunities.

6.4 Extensions and Future Work

Effects. Like Forster et al. [2019], the CBPV we consider doesn't include effects. However, by inspection, we can see that commuting conversions are effect safe. For instance, considering $\text{let } y \leftarrow (\text{let } x \leftarrow n_1 \text{ in } n_2) \text{ in } m$, if n_1 , n_2 , and m contained effects, they would be executed in that very order according to the evaluation semantics both before and after commutation.

Not all semantically equivalent computations preserve effect order. Given n_1 and n_2 where neither x nor y are free, $\text{let } x \leftarrow n_1 \text{ in let } y \leftarrow n_2 \text{ in } m$ is semantically equivalent to $\text{let } y \leftarrow n_2 \text{ in let } x \leftarrow n_1 \text{ in } m$, but would swap the order of effects in n_1 and n_2 . We believe that CC-normalization doesn't perform extraneous reordering transformations, but rigorously proving it effect safe requires incorporating effects into the logical relation.

Stack usage. Commuting conversions optimize stack usage. For the CBV lambda calculus, where the A-reductions of ANF are commuting conversions, Bowman [2024] views ANF as the A-normalization of monadic normal form (MNF), the syntactic form of the monadic meta-language by Moggi [1991] that binds intermediate computations. He shows that first taking A-reduction steps from MNF to ANF optimizes stack usage in comparison to evaluating the original MNF term under stack machine semantics.

Similarly, we can see that our CBPV commuting conversions may also reduce stack usage. For instance, considering $\text{let } y \leftarrow (\text{let } x \leftarrow n_1 \text{ in } n_2) \text{ in } m$, stack machine evaluation would first push $\text{let } y \leftarrow \square \text{ in } m$ onto the stack, followed by $\text{let } x \leftarrow \square \text{ in } n_2$, before evaluating n_1 and popping these two stack frames. In contrast, evaluating $\text{let } x \leftarrow n_1 \text{ in let } y \leftarrow n_2 \text{ in } m$ pushes $\text{let } x \leftarrow \square \text{ in let } y \leftarrow n_2 \text{ in } m$ onto the stack, evaluates n , pops this stack frame, then pushes $\text{let } y \leftarrow \square \text{ in } m$ next, using one fewer total stack frame.

Because our translation is a single-pass transformation that uses a continuation rather than a sequence of commuting conversions, which traverses the term each time, showing that CC-normalized computations optimize stack usage is nontrivial. It may require going through an intermediate big-step semantics to track maximum stack usage, similar to how Chen et al. [2025] use big-step to track space and time usage in their cost model of CBPV.

Compilation to assembly. One way to reap the benefits of the way commuting conversions unnest computations is to compile onwards to a lower-level assembly-like language. A potential compilation target is the lower-level "linearized CBPV" proposed by New [2019], which is a stack machine language with explicit push/pop and jump operations. We conjecture that CC-normalization prior to compilation to this language would optimize stack usage, and that our jumps can be compiled directly to its jumps.

7 Related Work

Forster et al. [2019] mechanize in Rocq a vast amount of metatheory for call-by-push-value, including normalization and observational equivalence. Our work builds on their design of logical equivalence between terms. They show a number of commuting conversions as semantic equivalences, namely commuting let-bound let expressions (Lemma 8.4, Equation 5) and function application of let expressions (Lemma 8.6, Equation 1), but are not comprehensive in listing all possible commuting conversions systematically as we have done.

Our join points and their typing judgements with a separate join point typing context are inspired by Maurer et al. [2017], who add join points and jumps to System F, and implement their system in the Haskell compiler GHC. We simplify the addition by only allowing jumps in tail position, while they permit jumps in evaluation contexts. They perform their optimizations equation by equation, which yields intermediate steps that require this permissiveness, such as the following.

$$\begin{aligned}
 (\text{join } j \ x = m \text{ in jump } j \ v) \ w &\Longrightarrow \text{join } j \ x = m \ w \text{ in } (\text{jump } j \ v) \ w \\
 &\Longrightarrow \text{join } j \ x = m \ w \text{ in jump } j \ v
 \end{aligned}$$

Because we present instead a single-pass algorithm, we never produce such intermediate steps that need to be typeable, so we are able to eliminate them from our syntax outright. Doing so also simplifies typing: their jumps need to be typeable with arbitrary types, which they implement using type polymorphism, while our jumps are always in tail position and fixed to the return type of the join point they jump to.

Their work on join points is in contrast to earlier work by Kennedy [2007], who argues for CPS instead of direct-style using second-class continuations, and from whom we borrow the terminology *CC-normalization*. Cong et al. [2019] combine ideas from both using control operators that bind second-class continuations, using them as join points. Their system allows for both direct style and continuation-passing style optimizations; this is not directly applicable to CBPV, which is already in direct style by virtue of binding intermediate computations. However, the corresponding dual of CBPV is stack-passing style [New 2023], and it would be interesting to see whether a similar technique could be applied. The Zydeco project is ongoing work implementing a compiler from CBPV to stack-passing style [Jiang et al. 2025].

8 Conclusion

In this paper, we looked at commuting conversions for call-by-push-value, which are syntactic transformations that preserve evaluation. By commuting evaluation contexts into tail positions, we unnest computations and expose inlining opportunities. We have identified a commuting conversion normal form for CBPV and presented a single-pass transformation into CCNF. To avoid code duplication without incurring additional closures, we used a separate join point construct that avoids creating new thunks. We have shown that the translation preserves not just typing but also evaluation, using a logical equivalence to prove that the translation of a term is equivalent to itself. Our results are entirely mechanized in Lean 4.

Acknowledgments

This work was supported by the National Science Foundation under Grant No. 2327738.

Data-Availability Statement

Our Lean 4 proof development is available as supplementary material [Chan 2026].

References

- William J. Bowman. 2024. A Low-Level Look at A-Normal Form. *Proc. ACM Program. Lang.* 8, OOPSLA2 (Oct. 2024), 165–191. doi:10.1145/3689717
- Jonathan Chan. 2026. *Artifact for “Commuting Conversions and Join Points for Call-By-Push-Value”*. doi:10.5281/zenodo.19288094
- Zhuo Chen, Johannes Åman Pohjola, and Christine Rizkallah. 2025. A Verified Cost Model for Call-by-Push-Value. In *Interactive Theorem Proving* (Reykjavik, Iceland) (ITP 2025). Springer International Publishing, Cham, Switzerland, Article 7, 19 pages. doi:10.4230/LIPIcs.ITP.2025.7
- Youyou Cong, Leo Osvald, Grégory M. Essertel, and Tiark Rumpf. 2019. Compiling with continuations, or without? whatever. *Proc. ACM Program. Lang.* 3, ICFP, Article 79 (July 2019), 28 pages. doi:10.1145/3341643
- Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean Theorem Prover (System Description). In *International Conference on Automated Deduction (Lecture Notes in Computer Science, Vol. 9195)*. Springer International Publishing, Cham, Switzerland, 378–388. doi:10.1007/978-3-319-21401-6_26

- Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The essence of compiling with continuations. In *ACM SIGPLAN'93 Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery, Albuquerque, NY, USA, 237–247. doi:10.1145/173262.155113
- Yannick Forster, Steven Schäfer, Simon Spies, and Kathrin Stark. 2019. Call-by-push-value in Coq: operational, equational, and denotational theory. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs (Cascais, Portugal) (CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 1180–131. doi:10.1145/3293880.3294097
- Dmitri Garbuzov, William Mansky, Christine Rizkallah, and Steve Zdancewic. 2018. *Structural Operational Semantics for Control Flow Graph Machines*. arXiv:1805.05400 [cs.PL] doi:10.48550/arXiv.1805.05400
- Yuchen Jiang, Max S. New, Tingting Ding, Runze Xue, Yuxuan Xia, and Nathan Varner. 2025. *zydeco-lang/zydeco: v0.2.2*. zydeco-lang. doi:10.5281/zenodo.15756916
- Andrew Kennedy. 2007. Compiling with continuations, continued. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (Freiburg, Germany) (ICFP '07)*. Association for Computing Machinery, New York, NY, USA, 177–190. doi:10.1145/1291151.1291179
- Paul Blain Levy. 2003. *Call-By-Push-Value: A Functional/Imperative Synthesis*. Springer Dordrecht, Dordrecht, Netherlands. doi:10.1007/978-94-007-0954-6
- Luke Maurer, Zena Ariola, Paul Downen, and Simon Peyton Jones. 2017. Compiling without continuations. In *ACM Conference on Programming Languages Design and Implementation (PLDI'17)*. Association for Computing Machinery, New York, NY, USA, 482–494. doi:10.1145/3062341.3062380
- Eugenio Moggi. 1991. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55–92. doi:10.1016/0890-5401(91)90052-4
- Greg Morrisett, Karl Crary, Neal Glew, and David Walker. 2002. Stack-based typed assembly language. *Journal of Functional Programming* 12, 1 (2002), 43–88. doi:10.1017/S0956796801004178
- Max S. New. 2019. From Call-by-push-value to Stack-Based TAL? Presentation at Syntax and Semantics of Low-Level Languages (LOLA 2019), Vancouver, BC, Canada. 2 pages. <https://maxsnew.com/docs/cbpv-stal-lola-2019.pdf>
- Max S. New. 2023. Compiling with Call-by-push-value. Presentation at Conference on Algebra and Coalgebra in Computer Science (CALCO 2023), Bloomington, IN, USA. 70 pages. <https://coalg.org/calco-mfps-2023/slides/compiling-with-cbpv-1.pdf>
- Christine Rizkallah, Dmitri Garbuzov, and Steve Zdancewic. 2018. A Formal Equational Theory for Call-By-Push-Value. In *Interactive Theorem Proving*, Jeremy Avigad and Assia Mahboubi (Eds.). Springer International Publishing, Cham, Switzerland, 523–541. doi:10.1007/978-3-319-94821-8_31
- Zachary J. Sullivan, Paul Downen, and Zena M. Ariola. 2023. Closure Conversion in Little Pieces. In *Proceedings of the 25th International Symposium on Principles and Practice of Declarative Programming (Lisboa, Portugal) (PPDP 2023)*. Association for Computing Machinery, New York, NY, USA, Article 10, 13 pages. doi:10.1145/3610612.3610622

Received 2025-10-10; accepted 2026-02-17